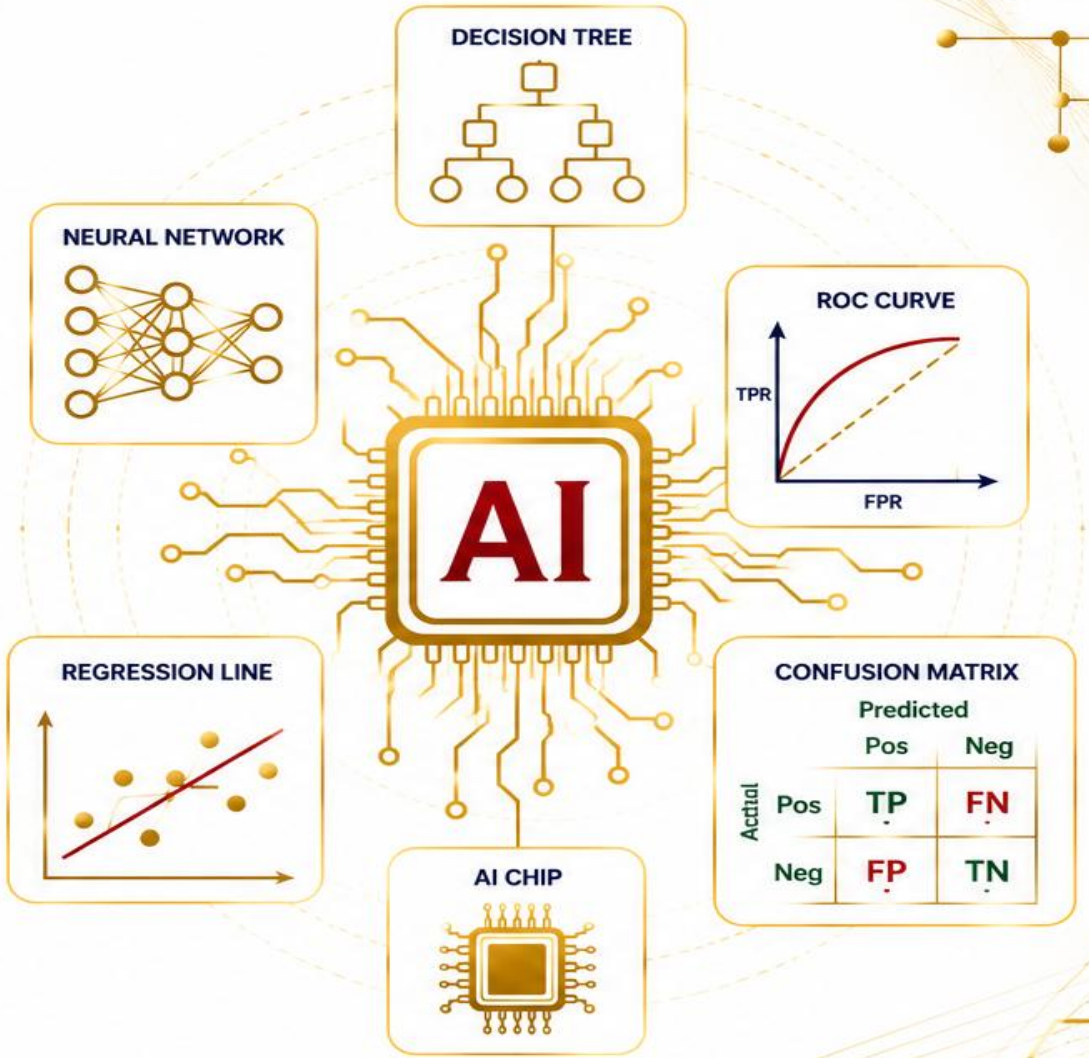
Explainable Artificial Intelligence

UNIT II

Model Visualization Techniques & Traditional Interpretable Algorithms

Dr Kiran Jasthi



Unit II – Outline

1.  Model Validation, Evaluation & Hyperparameters
2.  Model Selection and Visualization
3.  Classification Model Visualization
4.  Regression Model Visualization
5.  Clustering Model Visualization
6.  Interpretable Machine Learning Properties
7.  Traditional Interpretable Algorithms

DATA SCIENCE PIPELINE

- Typical pipeline: Problem Definition → Data Collection → EDA → Feature Engineering → Modeling → Evaluation → Deployment.

01 Problem Definition



- Understand business objectives.
- Define the problem statement.
- Identify success metrics.

02 Data Collection



- Identify data sources.
- Gather relevant data.
- Ensure data quality and reliability.

03 EDA (Exploratory Data Analysis)



- Understand the data.
- Handle missing values, outliers.
- Visualize distributions and relationships.
- Derive insights.

04 Feature Engineering



- Select relevant features.
- Create new features.
- Transform and encode variables.
- Remove redundant/ correlated features.

05 Modeling (Build & Train)



- Choose appropriate algorithms.
- Train models.
- Tune hyperparameters.
- Prefer interpretable models where possible.

06 Evaluation (Measure & Analyze)



- Evaluate model performance.
- Use appropriate metrics and visualizations.
- Validate and compare models.
- Analyze errors and bias.

07 Deployment



- Deploy the model to production.
- Monitor performance.
- Update and maintain the model.



Modeling and **Evaluation** are the core of model **interpretability.** They determine how well the model learns patterns and how reliably it performs on unseen data, enabling trustworthy and explainable AI.

Basic ML Model Families

CLASSIFICATION

- Predict a categorical class
- Logistic Regression
- Decision Tree
- KNN / SVM / Naive Bayes
- Example: Diabetes → Yes / No

REGRESSION

- Predict a continuous value
- Linear Regression
- Polynomial Regression
- Decision Tree / Random Forest
- Example: Insurance charge

CLUSTERING

- Group similar observations
- K-Means
- Hierarchical / DBSCAN
- Gaussian Mixture Model
- Example: Customer segments

Model Validation: Can the Model Generalize?

TRAINING DATA

- Learn model parameters
- Fit patterns from known examples

VALIDATION DATA

- Tune hyperparameters
- Select the best model
- Detect overfitting / underfitting

TEST DATA

- Unseen data
- Final performance estimate
- Do not tune on test data

GOOD GENERALIZATION

- Train and test performance are close

XAI : a reliable explanation requires a reliable model.

Hyperparameters, Tuning and Model Selection

HYPERPARAMETERS

- KNN → k
- Decision Tree → maximum depth
- SVM → C, kernel, gamma
- K-Means → K

TUNING

- Try candidate values
- Validate each configuration
- Grid Search / Random Search
- Cross-validation
- Select best configuration

MODEL SELECTION

- Compare performance
- Check generalization
- Consider complexity
- Consider interpretability
- Do not use accuracy alone

Best model = appropriate performance + generalization + acceptable complexity + interpretability.

Model → Validate → Tune → Test → Visualize → Interpret

A Reliable and Explainable ML Workflow

EDA + FEATURE ENGINEERED DATA (Labeled Data)
(Cleaned, transformed, feature engineered dataset ready for modeling)
Examples: Pima Diabetes (Classification) | Insurance Claims (Regression)

Training Data
(Model Learning)

Validation Data
(Tuning & Selection)

Test Data
(Final Evaluation)

Model
(Train on Training Data)

- Learn patterns
- Build predictive model

Validate → Tune
(Using Validation Data)

- Try different algorithms/ hyperparameters
- Select best model
- Avoid overfitting / underfitting

Test
(Using Test Data)

- Estimate real-world performance
- Measure generalization

Evaluation & Visualization

Classification

- Precision–Recall Curve
- ROC Curve
- Confusion Matrix
- Calibration Plot

Regression

- Residual Plot
- Actual vs Predicted
- Error Distribution

Clustering

- Silhouette Coefficient
- Cluster Plot
- Davies–Bouldin Index

Interpret
(Explain the Model)

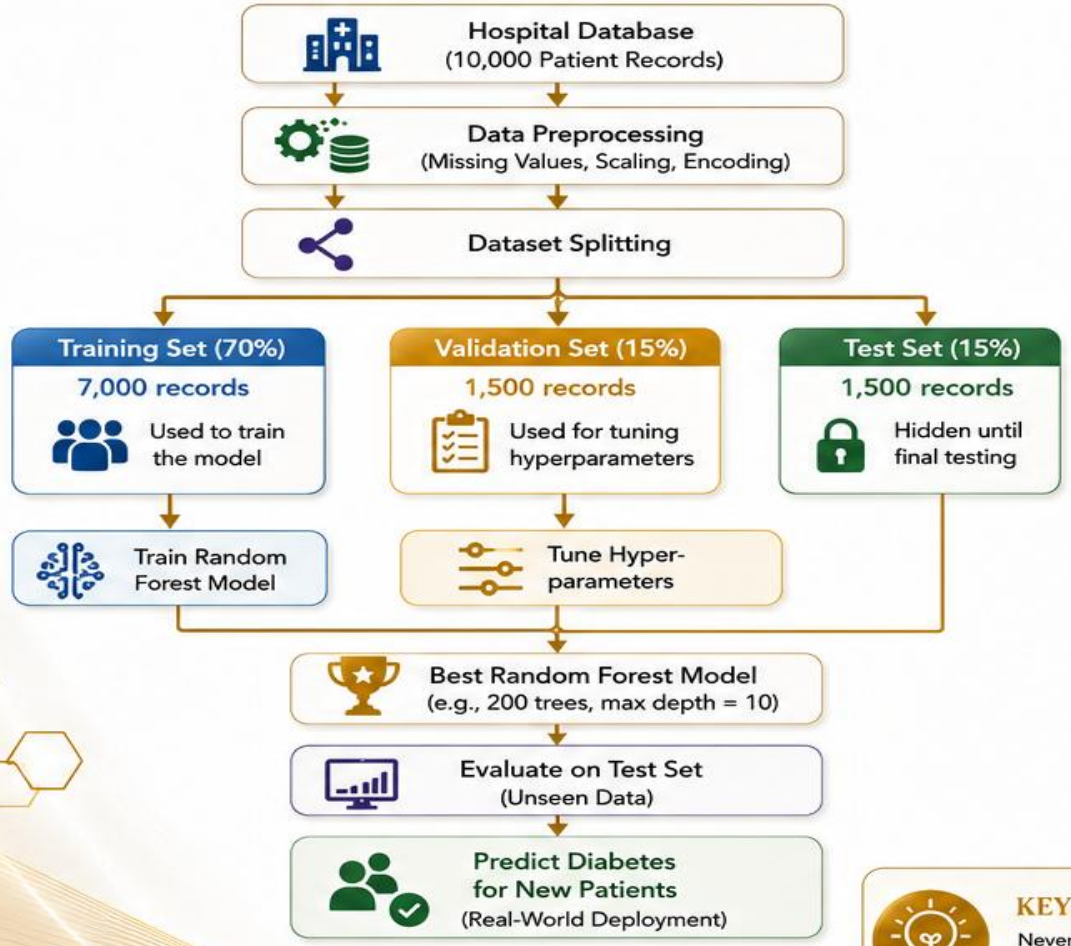
- Understand feature impact
- Identify model weaknesses
- Build trust
- Support better decisions

Goal: Build a model that not only performs well on unseen data but is also understandable and trustworthy.

Example: Predicting Diabetes Using Machine Learning

Objective: Develop a machine learning model to predict whether a patient has diabetes based on medical records.

REAL-WORLD WORKFLOW



WHAT HAPPENS IN EACH STAGE?

Training Set

- Learns relationships between patient features and diabetes
- Model learns from 7,000 patient records

Validation Set

- Finds the best hyperparameters
- Compares options (e.g., number of trees, tree depth)

Test Set

- Measures final performance on unseen data
- Predicts diabetes for 1,500 new patients

EXAMPLE RESULTS

Model	Description	Validation Accuracy	Test Accuracy
Random Forest (100 trees)	Baseline model with 100 decision trees	91.8%	90.9%
Random Forest (200 trees)	More trees for better learning	**93.4%**	**92.8%** ✓
Decision Tree	Single decision tree	88.1%	86.5%

Best Model: Random Forest with 200 trees because it performs best on both validation and unseen test data.

KEY TAKEAWAY

Never evaluate the model using the same data used for training. The test set must remain unseen until the final evaluation to obtain an unbiased estimate of real-world performance.

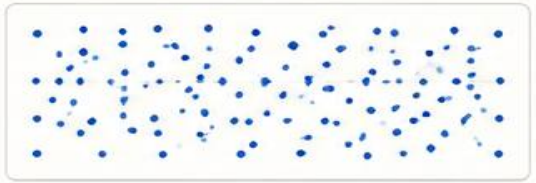
- Prevents Overfitting
- Ensures Generalization
- Better Real-World Performance

Data Splitting: How It Differs for Different Types of Data

The splitting strategy depends on the nature of the data to ensure unbiased evaluation on unseen data.

NORMAL (IID) DATA

Data points are independent and identically distributed.
Random splitting works well.



Random Shuffle Split



- Data is shuffled randomly
- Each subset has similar distribution

Example

Customer details, product ratings, genetic data, survey responses

Key Benefit

Simple, fast and gives reliable performance estimate for IID data.

TIME SERIES DATA

Observations are ordered in time.
Future data must not be used to predict the past.



Time-based (Chronological) Split



- Oldest data → Training
- Recent data → Validation
- Latest data → Test

Example

Stock prices, weather data, energy demand, sensor readings

Key Benefit

Preserves temporal order and prevents look-ahead bias (data leakage).

COMPUTER VISION DATA

Images from the same scene/object are highly correlated.
Split at the image (or group) level.



Group-wise / Stratified Split



- Keep images from the same scene/object in the same split
- Maintain class distribution (stratified)

Example

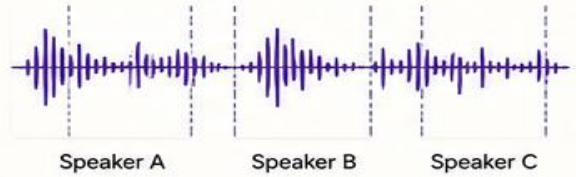
Image classification, object detection, medical images

Key Benefit

Prevents information leakage from similar images across splits.

SPEECH RECOGNITION DATA

Samples from the same speaker/session are highly correlated.
Split by speaker (or session), not by utterance.



Speaker-wise (or Session-wise) Split



- Entire speakers (or sessions) kept in one split
- Avoid same speaker in train & test

Example

Speech recognition, voice commands, speaker identification

Key Benefit

Ensures model is tested on unseen speakers for better generalization.

Key Takeaway

The way we split data has a huge impact on model evaluation.
Use a splitting strategy that respects the structure and dependencies in your data.

- Use random split for IID data.
- Use chronological split for time series data.
- Use group / image-level split for computer vision data.
- Use speaker / session-wise split for speech data.

Hyperparameter Optimization Techniques

Methods for Finding the Best Hyperparameter Values

Hyperparameter Optimization

Traditional Search

1. Manual Search
- Try values manually based on intuition
- Simple but inefficient

2. Grid Search
- Evaluate every possible combination
- Guarantees best in the search space
- Computationally expensive

3. Random Search
- Randomly sample combinations
- More efficient than Grid Search

Intelligent Search

4. Bayesian Optimization
- Model the objective function
- Choose next best set intelligently
- Faster convergence

5. Hyperband
- Allocate resources dynamically
- Early stop poor performing trials
- Very efficient

6. Optuna / Hyperopt
- Bayesian optimization with pruning
- Automatic & efficient
- Easy visualization

7. Evolutionary Algorithms
- Inspired by natural selection
- Good for complex, non-linear spaces

How Hyperparameter Optimization Works

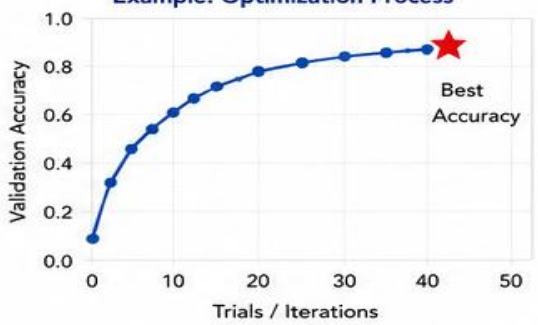
1. Initial Hyperparameters
Define a search space for hyperparameters.

2. Multiple Trials
Run models with different combinations.

3. Evaluate Performance
Measure using metrics (e.g., Accuracy, RMSE).

4. Best Model
Select the combination with the best performance.

Example: Optimization Process



Why Hyperparameter Optimization?

- Model performance heavily depends on hyperparameters.
- Good hyperparameters → better accuracy, stability, and generalization.
- Poor hyperparameters → overfitting, underfitting, slow convergence.

Common Hyperparameters in ML Models

- **Decision Trees / Random Forest:** max_depth, min_samples_split, n_estimators, max_features
- **SVM:** C, kernel, gamma
- **KNN:** n_neighbors, weights, metric
- **Neural Networks:** learning_rate, batch_size, epochs, dropout
- **XGBoost:** n_estimators, learning_rate, max_depth, subsample, colsample_bytree

Key Takeaway

Hyperparameter optimization is essential for building high-performing models.
Traditional **methods** (Manual, Grid, Random) are simple but limited.
Intelligent methods (Bayesian, Hyperband, Optuna/Hyperopt, Evolutionary) are more efficient and scalable.
The goal is to achieve the **best performance** with **minimum computational cost**.

- Faster Convergence
- Better Performance
- Efficient Resource Use
- Improved Generalization

2.Model Selection and Visualization

- Choosing the right model involves balancing bias, variance, and interpretability needs.
- Visualization tools help diagnose underfitting and overfitting before committing to a final model.
- Two key diagnostic plots: the validation curve(hyperparameter) and the learning curve (dataset size).

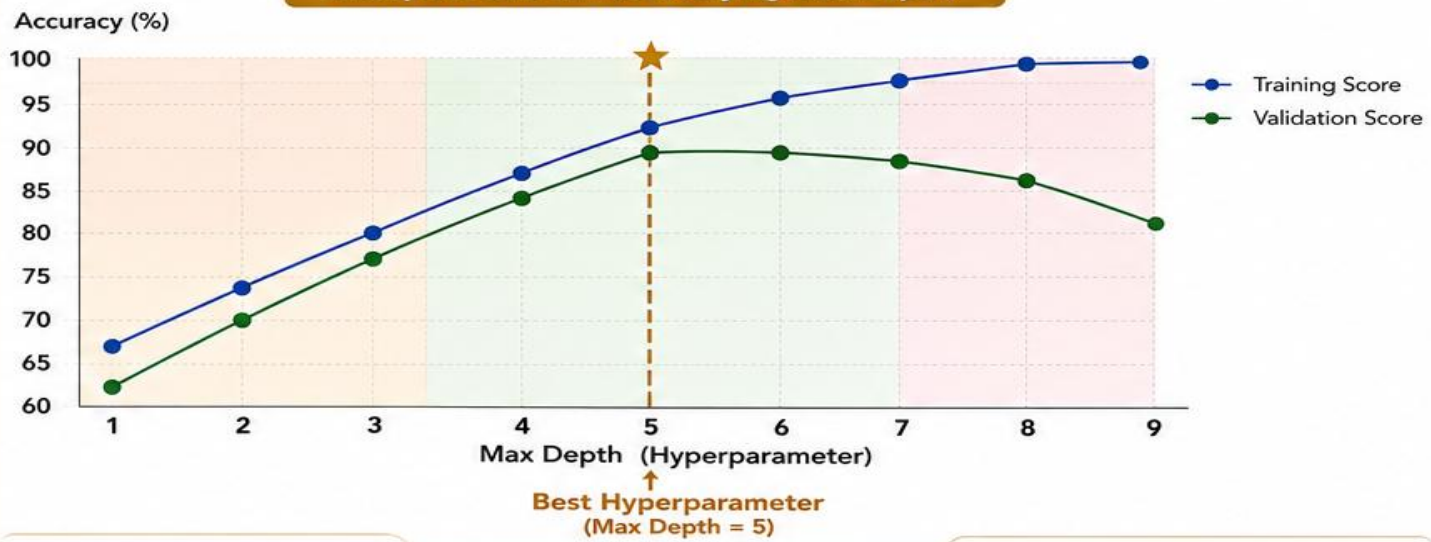
Validation Curve

- Plots training and validation scores as a function of a single hyperparameter (e.g., tree depth, regularization C).
- Helps identify the hyperparameter region where the model neither underfits nor overfits.
- Diverging train/validation curves at high model complexity signal overfitting.
- Both curves at low scores signal underfitting – the model is too simple for the task.

Validation Curve

Selecting the Optimal Hyperparameter for Better Generalization

Example: Decision Tree (Varying Max Depth)



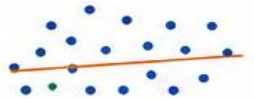
Underfitting
High bias, both training and validation accuracy are low.

Good Generalization
High accuracy with small gap between training and validation.

Overfitting
Training accuracy very high but validation accuracy decreases.

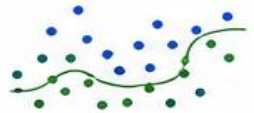
UNDERFITTING

- Model is too simple.
- High bias.
- Low training accuracy.
- Low validation accuracy.



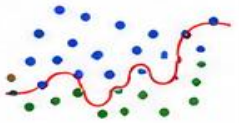
GOOD GENERALIZATION

- Balanced model complexity.
- High training accuracy.
- High validation accuracy.
- Best performance on unseen data.



OVERFITTING

- Model memorizes training data.
- Training accuracy very high.
- Validation accuracy decreases.
- Poor real-world performance.



Decision Tree Example

Max Depth	Training Accuracy (%)	Validation Accuracy (%)	Gap (Train – Val)
2	74	72	2
3	82	80	2
4	89	87	2
★ 5	93	91	2
6	97	89	8
8	100	84	16
9	100	82	18

How It Works

1. Choose a Hyperparameter
2. Train Model for Different Values
3. Compute Training & Validation Scores
4. Plot Scores (Validation Curve)
5. Select the Best Hyperparameter

Key Takeaway

A validation curve helps us understand how a model's performance changes with a hyperparameter. Choose the value where validation score is highest and the gap with training score is small.

Better Generalization | Avoid Overfitting | Optimad Parameters | Improved Performance

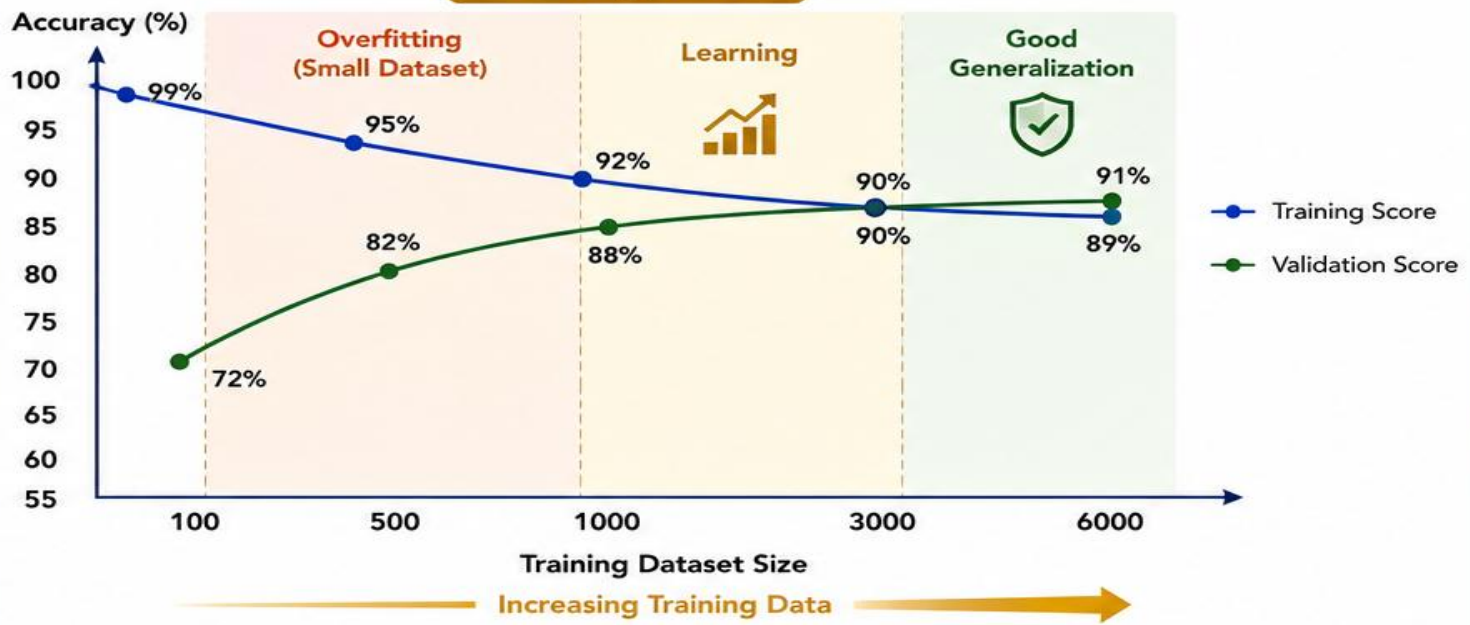
Learning Curve

- Plots training and validation scores as a function of training set size.
- A large persistent gap between training and validation scores indicates high variance (overfitting) – more data or regularization may help.
- Both curves converging to a low score indicates high bias (underfitting) – a more expressive model or richer features may help.
- Useful for deciding whether collecting more data is worth the cost.

Learning Curve

Understanding Model Performance as the Training Dataset Increases

Learning Curve



1. Small Training Dataset

- Few samples
- Model memorizes data
- High training accuracy
- Low validation accuracy
- **Overfitting**

2. Increasing Dataset

- More representative data
- Better learning
- Validation accuracy improves
- Reduced overfitting

3. Sufficient Dataset

- Curves converge
- Stable performance
- Good generalization
- Reliable prediction

More Data
Better Learning

Example: House Price Prediction

Training Samples	Training Accuracy (%)	Validation Accuracy (%)	Gap (Train – Val)
100	99%	73%	26%
500	95%	82%	13%
1000	92%	88%	4%
3000	90%	90%	0%
6000 ★	89%	91%	-2%

How It Works

1. Collect More Data
2. Train the Model
3. Evaluate Training & Validation Scores
4. Observe Learning Curve Trend
5. Achieve Better Generalization

Key Insight: As the training dataset increases, the model learns more generalized patterns.
The gap between the training and validation curves decreases, resulting in better performance on unseen data.

Tools & Libraries for Validation and Hyperparameter Optimization

scikit-learn

Model validation and hyperparameter tuning

Key Features

- cross_val_score()
- GridSearchCV
- RandomizedSearchCV

Example

```
from sklearn.model_selection import
GridSearchCV, cross_val_score
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
param_grid = {'n_estimators': [100, 200, 300],
              'max_depth': [None, 10, 20]}

grid = GridSearchCV(model, param_grid,
            cv=5, scoring='accuracy')
```

Standard, reliable and widely used for model evaluation and tuning.

Yellowbrick

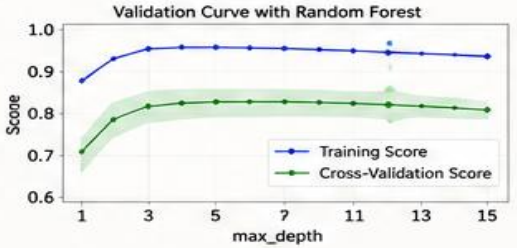
Visualize ML

Visual diagnostics built on top of scikit-learn

Key Features

- Validation Curves
- Learning Curves
- ROC Curve
- Residual Plot
- Confusion Matrix

Example – Validation Curve



Visualize model behavior and detect overfitting/underfitting.

OPTUNA / hyperopt

Automated hyperparameter optimization

Key Features

- Bayesian Optimization
- Efficient parameter search
- Pruning unpromising trials
- Search trajectory visualization

Example – Optimization History



Find optimal hyperparameters faster with intelligent search.

mlflow

Experiment tracking and comparison

Key Features

- Log parameters
- Track metrics
- Compare experiments
- Model versioning
- Visualize results

Example – MLflow Runs

Runs (20) Filter

Run Name	RMSE	R^2 Score	Params
run_1	0.512	0.876	{...}
run_2	0.436	0.904	{...}
run_3	0.498	0.884	{...}
run_4	0.621	0.832	{...}

Track, compare and reproduce ML experiments in one place.

Key Takeaway

scikit-learn helps you validate and tune models, **Yellowbrick** explains model performance visually, **Optuna/Hyperopt** automates the search for best parameters, and **MLflow** tracks the entire ML experiment lifecycle.

Classification Model Visualization: Overview

- Visual diagnostics for classifiers go beyond a single accuracy number to reveal where and how a model errs.
- Key tools: confusion matrix, classification report, ROC/AUC, precision-recall curve (PRC), discrimination threshold plots.

Confusion Matrix and Classification Report

- Confusion matrix: cross-tabulates predicted vs. actual classes, exposing true positives, false positives, true negatives, false negatives.
- Reveals systematic error patterns (e.g., consistently confusing two specific classes) – valuable for explaining model weaknesses.
- Classification report summarizes precision, recall, F1-score, and support per class, enabling per-class interpretability of performance.

CONFUSION MATRIX USING SPAM EMAIL DETECTION

SPAM DETECTION RESULTS (Threshold = 0.50)

#	Email Preview	Actual (Label)	Spam Probability	Prediction (Threshold = 0.50)	Result
1	Win ₹10 Lakhs Lottery!!! Claim Now	Spam	0.96 — 96%	Spam	TP
2	Office Meeting at 3 PM	Ham	0.08 — 8%	Ham	TN
3	Limited Time Discount – 80% OFF	Spam	0.87 — 87%	Spam	TP
4	Bank OTP for A/C XXXXX1234	Ham	0.72 — 72%	Spam	FP
5	Family Photos from Vacation	Ham	0.15 — 15%	Ham	TN
6	Claim Your Prize – Act Now!	Spam	0.61 — 61%	Spam	TP
7	Electricity Bill for May 2024	Ham	0.22 — 22%	Ham	TN
8	Free Crypto Investment Tips	Spam	0.43 — 43%	Ham	FN
9	Amazon Great Indian Sale	Spam	0.79 — 79%	Spam	TP
10	Project Report – Final Version	Ham	0.18 — 18%	Ham	TN

THRESHOLD VISUALIZATION



Probability < 0.50 → Not Spam (Ham) → Go to Inbox

Probability ≥ 0.50 → Spam → Move to Spam Folder

DATASET SUMMARY

Total Emails	Spam (Actual)	Ham (Actual)
10	5	5

CONFUSION MATRIX (Threshold = 0.50)

Predicted \ Actual (Ground Truth)	Spam (Positive)	Ham (Negative)
Spam (Positive)	4 (TP)	1 (FP)
Ham (Negative)	1 (FN)	4 (TN)

- **TP (True Positive)** — Spam email correctly identified as Spam
- **FP (False Positive)** — Ham email incorrectly identified as Spam
- **FN (False Negative)** — Spam email incorrectly identified as Ham
- **TN (True Negative)** — Ham email correctly identified as Ham

- TP = 4 — Correctly predicted Spam
- TN = 4 — Correctly predicted Ham
- FP = 1 — Ham predicted as Spam (False Alarm)
- FN = 1 — Spam predicted as Ham (Missed Spam)

Total Emails = 10

EFFECT OF DIFFERENT THRESHOLD VALUES
ON CONFUSION MATRIX (SPAM EMAIL DETECTION)

Total Emails	Actual Spam	Actual Ham
10	5	5

Prediction Rule: If Probability ≥ Threshold → Spam
If Probability < Threshold → Ham

Threshold	Predicted Spam	Predicted Ham	TP (Spam → Spam)	TN (Ham → Ham)	FP (Ham → Spam)	FN (Spam → Ham)
0.30	6	4	5	4	1	0
0.40	6	4	5	4	1	0
0.50	5	5	4	4	1	1
0.60	5	5	4	4	1	1
0.70	4	6	3	4	1	2
0.80	2	8	2	5	0	3
0.90	1	9	1	5	0	4

KEY OBSERVATIONS

LOW THRESHOLD (0.30 – 0.40)
- Detects almost all spam emails
- Recall is very high (100%)
- More genuine emails are wrongly classified as spam (higher FP)

MEDIUM THRESHOLD (0.50 – 0.60)
- Balanced trade-off between precision and recall
- TP = 4, TN = 4, FP = 1, FN = 1
- Good overall performance

HIGH THRESHOLD (0.80 – 0.90)
- Very few false positives (High precision)
- But many spam emails are missed (higher FN)
- Recall is low

0.30 — 0.40 — 0.50 — 0.60 — 0.70 — 0.80 — 0.90

High Recall
More False Positives

Balanced Performance
(Trade-off Zone)

High Precision
More False Negatives

PERFORMANCE METRIC CALCULATION
(SPAM EMAIL DETECTION)

THRESHOLD = 0.50

CONFUSION MATRIX

		Actual: Spam (Positive)	Actual: Ham (Negative)
Predicted	Spam (Positive)	**TP = 4** (Correctly predicted Spam)	**FP = 1** (Ham predicted as Spam)
Predicted	Ham (Negative)	**FN = 1** (Spam predicted as Ham)	**TN = 4** (Correctly predicted Ham)

Total Emails	Actual Spam	Actual Ham
10	5	5

WHAT DO THESE VALUES MEAN?

TP (4) — Correctly predicted Spam



Actual Spam → Predicted Spam

FP (1) — Ham predicted as Spam



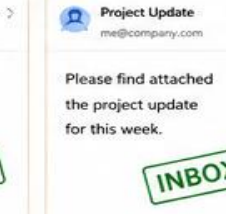
Actual Ham → Predicted Spam

FN (1) — Spam predicted as Ham



Actual Spam → Predicted Ham

TN (4) — Correctly predicted Ham



Actual Ham → Predicted Ham

METRIC CALCULATIONS (STEP-BY-STEP)

1. ACCURACY

Proportion of total correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$= (4 + 4) / (4 + 4 + 1 + 1) = 8 / 10$

$= 0.80\ (80\%)$

2. PRECISION

Proportion of predicted Spam that are actually Spam.

$$\text{Precision} = \frac{TP}{TP + FP}$$

$= 4 / (4 + 1) = 4 / 5$

$= 0.80\ (80\%)$

3. RECALL (SENSITIVITY)

Proportion of actual Spam that are correctly detected.

$$\text{Recall} = \frac{TP}{TP + FN}$$

$= 4 / (4 + 1) = 4 / 5$

$= 0.80\ (80\%)$

4. SPECIFICITY

Proportion of actual Ham that are correctly identified.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

$= 4 / (4 + 1) = 4 / 5$

$= 0.80\ (80\%)$

5. F1-SCORE

Harmonic mean of Precision and Recall.

$$\text{F1-Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

$= 2 \times (0.80 \times 0.80) / (0.80 + 0.80)$

$= 1.28 / 1.60$

$= 0.80\ (80\%)$

SUMMARY OF RESULTS

- Accuracy = 80%
- Precision = 80%
- Recall = 80%
- Specificity = 80%
- F1-Score = 80%

★ With threshold = 0.50, the model achieves balanced performance across all key metrics.

All metrics range from 0 to 1 (or 0% to 100%). Higher values indicate better performance.

PERFORMANCE METRICS FOR DIFFERENT THRESHOLD VALUES
(SPAM EMAIL DETECTION)

Total Emails	Actual Spam	Actual Ham
10	5	5

PREDICTION RULE

If Probability ≥ Threshold → Predict Spam

If Probability < Threshold → Predict Ham



Threshold	CONFUSION MATRIX (Counts)				PERFORMANCE METRICS				
	TP (Spam → Spam)	TN (Ham → Ham)	FP (Ham → Spam)	FN (Spam → Ham)	Accuracy (TP+TN) / N	Precision TP / (TP+FP)	Recall TP / (TP+FN)	Specificity TN / (TN+FP)	F1-Score 2PR / (P+R)
0.30	5	4	1	0	(5+4) / 10 = 0.90 (90%)	5 / (5+1) = 0.83 (83.3%)	5 / (5+0) = 1.00 (100%)	4 / (4+1) = 0.80 (80%)	2×0.83×1.00/(0.83+1.00) = 0.91 (90.9%)
0.40	5	4	1	0	(5+4) / 10 = 0.90 (90%)	5 / (5+1) = 0.83 (83.3%)	5 / (5+0) = 1.00 (100%)	4 / (4+1) = 0.80 (80%)	2×0.83×1.00/(0.83+1.00) = 0.91 (90.9%)
0.50	4	4	1	1	(4+4) / 10 = 0.80 (80%)	4 / (4+1) = 0.80 (80%)	4 / (4+1) = 0.80 (80%)	4 / (4+1) = 0.80 (80%)	2×0.80×0.80/(0.80+0.80) = 0.80 (80%)
0.60	4	4	1	1	(4+4) / 10 = 0.80 (80%)	4 / (4+1) = 0.80 (80%)	4 / (4+1) = 0.80 (80%)	4 / (4+1) = 0.80 (80%)	2×0.80×0.80/(0.80+0.80) = 0.80 (80%)
0.70	3	4	1	2	(3+4) / 10 = 0.70 (70%)	3 / (3+1) = 0.75 (75%)	3 / (3+2) = 0.60 (60%)	4 / (4+1) = 0.80 (80%)	2×0.75×0.60/(0.75+0.60) = 0.67 (66.7%)
0.80	2	5	0	3	(2+5) / 10 = 0.70 (70%)	2 / (2+0) = 1.00 (100%)	2 / (2+3) = 0.40 (40%)	5 / (5+0) = 1.00 (100%)	2×1.00×0.40/(1.00+0.40) = 0.57 (57.1%)
0.90	1	5	0	4	(1+5) / 10 = 0.60 (60%)	1 / (1+0) = 1.00 (100%)	1 / (1+4) = 0.20 (20%)	5 / (5+0) = 1.00 (100%)	2×1.00×0.20/(1.00+0.20) = 0.33 (33.3%)



KEY INSIGHTS

LOW THRESHOLD (0.30 – 0.40)
- Detects almost all spam
- Recall is very high (100%)
- But more genuine emails are wrongly classified as spam (higher FP)

MEDIUM THRESHOLD (0.50 – 0.60)
- Balanced trade-off between precision and recall
- All key metrics = 80%
- Good overall performance

HIGHER THRESHOLD (0.70)
- Fewer false positives
- But some spam emails are missed (Recall drops to 60%)

VERY HIGH THRESHOLD (0.80 – 0.90)
- No false positives (Precision & Specificity = 100%)
- But many spam emails are missed (Recall drops to 20–40%)

N = 10 (Total Emails) | All metric values range from 0 to 1 (or 0% to 100%). Higher values indicate better performance.

Confusion Matrix and Classification Report

Understand where the model is right, where it is wrong, and why it matters.

Example Problem

Use Case: Email Spam Detection

Goal: Classify emails as Spam (1) or Not Spam (0)



Test Set Summary

- Total Emails Tested: 10,000
- Actual Spam (1): 4,000 (40%)
- Actual Not Spam (0): 6,000 (60%)

1. Confusion Matrix (Test Set Results)

Actual \ Predicted	Not Spam (0)	Spam (1)
Not Spam (0)	TN 5,420 (54.2%)	FP 580 (5.8%)
Spam (1)	FN 680 (6.8%)	TP 3,320 (33.2%)

Where:

- TP: True Positives (Correctly predicted Spam)
- TN: True Negatives (Correctly predicted Not Spam)
- FP: False Positives (Not Spam predicted as Spam)
- FN: False Negatives (Spam predicted as Not Spam)

2. Key Metrics (Calculated from Confusion Matrix)

Accuracy
= (TP + TN) / Total
= (3,320 + 5,420) / 10,000
= 8,740 / 10,000
= 0.874 → **87.4%**

Precision (Spam class)
= TP / (TP + FP)
= 3,320 / (3,320 + 580)
= 3,320 / 3,900
= 0.851 → **85.1%**

Recall / Sensitivity (Spam class)
= TP / (TP + FN)
= 3,320 / (3,320 + 680)
= 3,320 / 4,000
= 0.830 → **83.0%**

Specificity (Not Spam class)
= TN / (TN + FP)
= 5,420 / (5,420 + 580)
= 5,420 / 6,000
= 0.903 → **90.3%**

F1-Score (Spam class)
= 2 × (Precision × Recall) / (Precision + Recall)
= 2 × (0.851 × 0.830) / (0.851 + 0.830)
= 2 × 0.70633 / 1.681
= 0.840 → **84.0%**



Takeaway: Confusion Matrix shows the big picture, Classification Report gives the detailed performance.
Always look beyond accuracy!

3. Classification Report (Per-Class Performance)

Class	Precision	Recall (Sensitivity)	F1-Score	Support
0 (Not Spam)	0.889	0.903	0.896	6,000
1 (Spam)	0.851	0.830	0.840	4,000
Accuracy	–	–	**0.874**	**10,000**
Macro Avg	0.870	0.866	0.868	10,000
Weighted Avg	0.874	0.874	0.873	10,000

4. Macro Average vs Weighted Average (How Calculated)

Macro Average (simple everage of classes)

- Macro Precision = (0.889 + 0.851) / 2 = **0.870**
- Macro Recall = (0.903 + 0.830) / 2 = **0.866**
- Macro F1-Score = (0.896 + 0.840) / 2 = **0.868**

Treats all classes equally, regardless of support.

Weighted Average (weighted by support)

- Weighted Precision = (0.889×6,000 + 0.851×4,000) / 10,000
  = (5,334 + 3,404) / 10,000 = **0.874**
- Weighted Recall = (0.903×6,000 + 0.830×4,000) / 10,000
  = (5,418 + 3,320) / 10,000 = **0.874**
- Weighted F1-Score = (0.896×6,000 + 0.840×4,000) / 10,000
  = (5,376 + 3,360) / 10,000 = **0.873**

Considers class imbalance using support (actual count).

5. Why It Matters?



False Negatives (680)
Important spam emails land in inbox → risk!



False Positives (580)
Important emails marked as spam → user impact!



Use metrics to tune threshold and improve model.

Accuracy alone can be misleading, especially with imbalanced data.
Always use multiple metrics to understand model behavior.

EXERCISE: AI-BASED DIABETES DETECTION

Use the dataset below to perform classification using a threshold of 0.50

Hospital
Apollo Health Centre

AI Model
Diabetes Risk Prediction

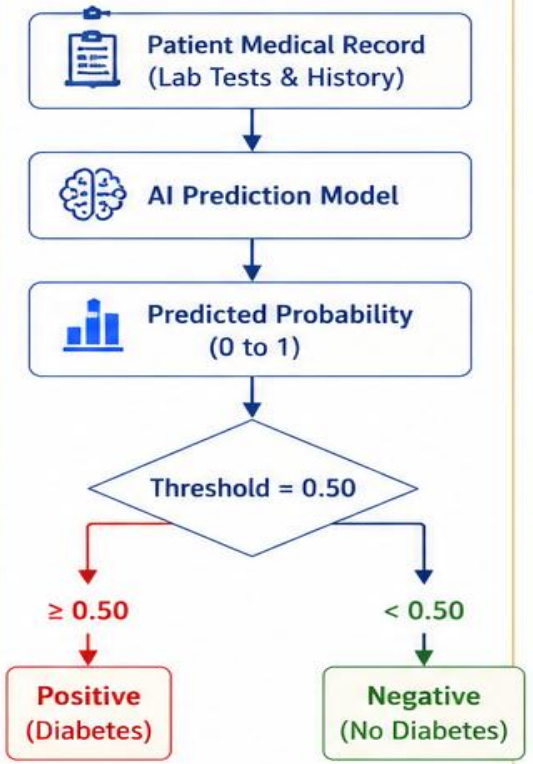
Decision Threshold
0.50

- If Probability ≥ 0.50 → Positive
- If Probability < 0.50 → Negative

PATIENT DATASET

Patient ID	Age (Years)	Fasting Glucose (mg/dL)	HbA1c (%)	Actual Condition	AI Probability (Diabetes)
P1	52	182	8.6	Positive	0.94
P2	34	96	5.3	Negative	0.15
P3	48	170	8.1	Positive	0.88
P4	46	132	6.4	Negative	0.72
P5	29	91	5.2	Negative	0.21
P6	54	165	7.8	Positive	0.67
P7	37	108	5.8	Negative	0.33
P8	45	126	6.5	Positive	0.48
P9	58	176	8.3	Positive	0.79
P10	27	89	5.1	Negative	0.09

AI DECISION WORKFLOW



STUDENT EXERCISE

1. **Predict** each patient as **Positive** or **Negative** using **Threshold = 0.50.**
2. Calculate TP, TN, FP, FN.
3. Construct the Confusion Matrix.
4. Compute the following performance metrics:
   - Accuracy
   - Precision
   - Recall (Sensitivity)
   - Specificity
   - F1-Score

Note: This exercise helps you understand how threshold selection affects classification results and evaluation metrics.

ROC and AUC

- ROC (Receiver Operating Characteristic) curve plots True Positive Rate vs. False Positive Rate across all classification thresholds.
- AUC (Area Under Curve) summarizes overall discriminative ability in a single number (0.5 = random, 1.0 = perfect).
- Helps explain model behavior across the entire operating range, not just at a single default threshold.
- Useful for comparing multiple models' ranking ability independent of class imbalance.

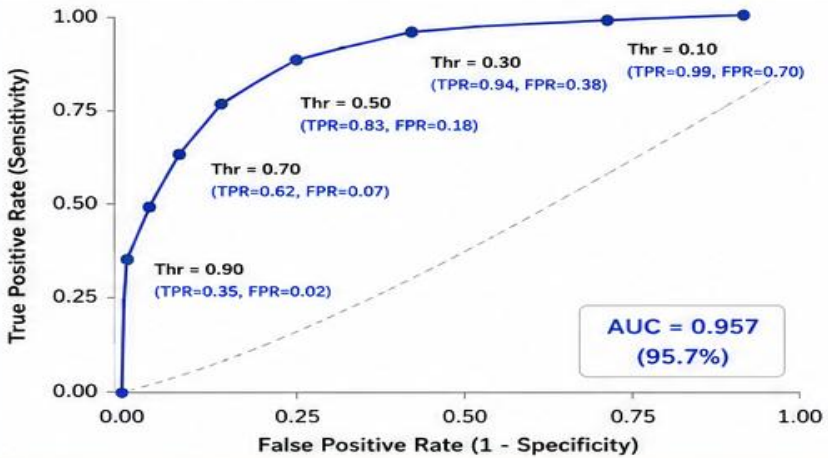
ROC and AUC – Explained with Real Example

Example Problem: Email Spam Detection

- Total Emails Tested: 10,000
- Actual Spam (1): 4,000 (40%)
- Actual Not Spam (0): 6,000 (60%)



1. ROC Curve (Test Set)



How to Read:

- Y-axis (TPR): Ability to catch actual spam.
- X-axis (FPR): How many not-spam emails are wrongly marked as spam.
- Curve closer to top-left corner = better model.

2. Data Points Used to Draw ROC Curve

Threshold (Probability cut-off)	TP (Spam correctly identified)	FP (Not Spam wrongly identified)	FN (Spam missed)	TN (Not Spam correctly identified)	TPR (Sensitivity) = TP / (TP+FN)	FPR (1-Specificity) = FP / (FP+TN)
1.00	0	0	4,000	6,000	0.00	0.00
0.90	1,400	120	2,600	5,880	0.35	0.02
0.70	2,480	420	1,520	5,580	0.62	0.07
0.50	3,320	1,080	680	4,920	0.83	0.18
0.30	3,760	2,280	240	3,720	0.94	0.38
0.10	3,960	4,200	40	1,800	0.99	0.70
0.00	4,000	6,000	0	0	1.00	1.00

3. AUC Calculation (Trapezoidal Rule)

Between Points (FPR_{i-1}, TPR_{i-1}) → (FPR_i, TPR_i)	FPR (x)	TPR (y)	$\Delta x = x_i - x_{i-1}$	Average y = $(y_i + y_{i-1}) / 2$	Area = $\Delta x \times$ Average y
(0.00, 0.00) → (0.02, 0.35)	0.00 → 0.02	0.00 → 0.35	0.02	(0.00 + 0.35)/2 = 0.175	0.02 × 0.175 = 0.0035
(0.02, 0.35) → (0.07, 0.62)	0.02 → 0.07	0.35 → 0.62	0.05	(0.35 + 0.62)/2 = 0.485	0.05 × 0.485 = 0.0243
(0.07, 0.62) → (0.18, 0.83)	0.07 → 0.18	0.62 → 0.83	0.11	(0.62 + 0.83)/2 = 0.725	0.11 × 0.725 = 0.0798
(0.18, 0.83) → (0.38, 0.94)	0.18 → 0.38	0.83 → 0.94	0.20	(0.83 + 0.94)/2 = 0.885	0.20 × 0.885 = 0.1770
(0.38, 0.94) → (0.70, 0.99)	0.38 → 0.70	0.94 → 0.99	0.32	(0.94 + 0.99)/2 = 0.965	0.32 × 0.965 = 0.3088
(0.70, 0.99) → (1.00, 1.00)	0.70 → 1.00	0.99 → 1.00	0.30	(0.99 + 1.00)/2 = 0.995	0.30 × 0.995 = 0.2985

Total AUC (Area Under Curve) = 0.0035 + 0.0243 + 0.0798 + 0.1770 + 0.3088 + 0.2985 = **0.8919 ≈ 0.892 (89.2%)**

4. Threshold Explanation (How It Affects the Model)

High Threshold (Closer to 1.0) — Trade-off — **Low Threshold (Closer to 0.0)**

Example: Threshold = 0.90

- TPR = 0.33 (35% spam caught)
- FPR = 0.02 (2% not-spam wrongly marked)
- Very few false positives
- Safer, but may miss spam

Example: Threshold = 0.50

- TPR = 0.83 (83% spam caught)
- FPR = 0.18 (18% not-spam wrongly marked)
- Balanced trade-off
- Good balance between catching spam and false alarms

Example: Threshold = 0.10

- TPR = 0.99 (99% spam caught)
- FPR = 0.70 (70% not-spam wrongly marked)
- Catches almost all spam
- Many false alarms

5. Interpretation



AUC = 0.957 (95.7%)

- ✅ AUC of 0.957 means the model has a 95.7% chance of ranking a random spam email higher than a random not-spam email.
- ✅ The model performs very well across all thresholds.
- ✅ We can choose a threshold based on business needs and cost of errors.
  - If false positives are costly (e.g., productivity loss) → use higher threshold (e.g., 0.70 – 0.90).
  - If missing spam is costly (e.g., security risk) → use lower threshold (e.g., 0.10 – 0.30).

6. Common AUC Interpretation (Rule of Thumb)

AUC Range	Interpretation
0.90 – 1.00	Excellent
0.80 – 0.90	Good
0.70 – 0.80	Fair
0.60 – 0.70	Poor
0.50 – 0.60	No better than random



Takeaway

ROC curve shows the trade-off between catching spam and raising false alarms across all thresholds. AUC summarizes this performance in a single number – higher is better.

Precision-Recall Curve (PRC)

- Plots precision vs. recall across thresholds – particularly informative under class imbalance, where ROC can be overly optimistic.
- A model with high area under the PRC maintains good precision even at high recall.
- Helps stakeholders understand the cost trade-off between false positives and false negatives explicitly.

Precision–Recall Curve (PRC) – Explained with Real Example

Understand how Precision and Recall change with different thresholds and why PRC is ideal for imbalanced data.

Example Problem

Use Case: Email Spam Detection

Goal: Classify emails as Spam (1) or Not Spam (0)



Test Set Summary

- Total Emails Tested: 10,000
- Actual Spam (1): 4,000 (40%)
- Actual Not Spam (0): 6,000 (60%)

1. Confusion Matrix (At Threshold = 0.50)

Actual \ Predicted	Not Spam (0)	Spam (1)
Not Spam (0)	TN 4,920 (49.2%)	FP 1,080 (10.8%)
Spam (1)	FN 680 (6.8%)	TP 3,320 (33.2%)

Where:

- **TP:** True Positives (Correctly predicted Spam)
- **TN:** True Negatives (Correctly predicted Not Spam)
- **FP:** False Positives (Not Spam predicted as Spam)
- **FN:** False Negatives (Spam predicted as Not Spam)

2. Key Metrics (Calculated at Threshold = 0.50)

Accuracy
= (TP + TN) / Total
= (3,320 + 4,920) / 10,000
= 8,240 / 10,000
= 0.824 → **82.4%**

Precision (Spam class)
= TP / (TP + FP)
= 3,320 / (3,320 + 1,080)
= 3,320 / 4,400
= 0.755 → **75.5%**

Recall / Sensitivity (Spam class)
= TP / (TP + FN)
= 3,320 / (3,320 + 680)
= 3,320 / 4,000
= 0.830 → **83.0%**

Specificity (Not Spam class)
= TN / (TN + FP)
= 4,920 / (4,920 + 1,080)
= 4,920 / 6,000
= 0.820 → **82.0%**

F1-Score (Spam class)
= 2 × (Precision × Recall) / (Precision + Recall)
= 2 × (0.755 × 0.830) / (0.755 + 0.830)
= 2 × 0.6266 / 1.585
= 0.790 → **79.0%**

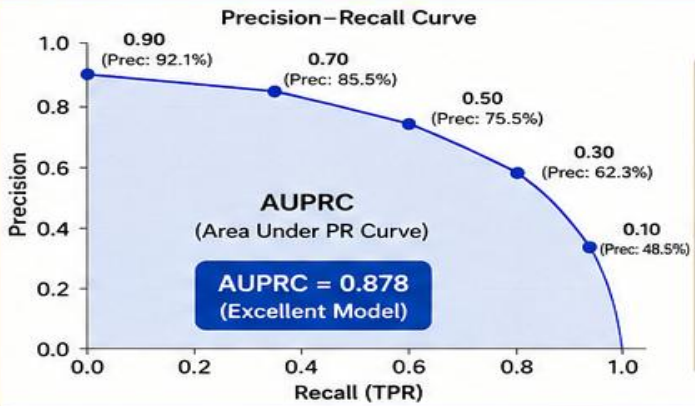
3. Threshold Effect on Precision and Recall

Threshold (Probability Cut-off)	TP (Spam Correctly Identified)	FP (Not Spam wrongly identified)	Precision TP / (TP + FP)	Recall (TPR) TP / (TP + FN)
0.90	1,400	120	92.1%	35.0%
0.70	2,480	420	85.5%	62.0%
0.50	3,320	1,080	75.5%	83.0%
0.30	3,760	2,280	62.3%	94.0%
0.10	3,960	4,200	48.5%	99.0%

High Threshold (Strict) → Low Threshold (Liberal)

As threshold decreases → Recall increases, but Precision decreases.

4. Precision–Recall Curve (PRC)



Higher curve (closer to top-right) indicates better performance.

AUPRC summarizes model performance for imbalanced data.

PRC focuses on the quality of positive predictions (Spam).

5. Why PRC Matters?

False Negatives (missed spam) → important land in inbox → risk!

False Positives (legit emails marked as spam) → poor user experience!

PRC shows the trade-off between catching more spam and avoiding false alarms.

Choose threshold based on business cost trade-off.

6. Interpretation (How to Choose Threshold)

Business Goal	Preferred Threshold	Effect	Example Use Case
Minimize False Negatives (Catch more spam)	Low (e.g., 0.10 – 0.30)	Recall High ↑ Precision Low ↓	Security, fraud prevention, phishing emails
Balanced Trade-off	Medium (e.g., 0.40 – 0.60)	Precision & Recall Balanced	General purpose email filters
Minimize False Positives (Avoid wrong spam)	High (e.g., 0.70 – 0.90)	Precision High ↑ Recall Low ↓	Important business emails, customer communications

Key Takeaway: The Precision–Recall Curve (PRC) is a powerful tool for evaluating models on imbalanced data like spam detection. It helps you select the best threshold to balance catching spam and minimizing wrong classifications.

Discrimination Thresholds

- Visualizes precision, recall, F1, and queue rate as functions of the classification threshold.
- Helps practitioners choose an operating threshold aligned with business/domain constraints (e.g., minimizing false negatives in disease screening).
- Makes explicit the otherwise hidden decision of “where to draw the line” between classes.

Discrimination Thresholds

Visualizes precision, recall, F1, and queue rate as functions of the classification threshold.

Example: Email Spam Detection

- Total Emails Tested: 10,000
- Actual Spam (1): 4,000 (40%)
- Actual Not Spam (0): 6,000 (60%)



1. Confusion Matrix (Test Set Results) at Threshold = 0.50

Actual \ Predicted	Not Spam (0)	Spam (1)
Not Spam (0)	TN 4,920 (49.2%)	FP 1,080 (10.8%)
Spam (1)	FN 680 (6.8%)	TP 3,320 (33.2%)

Where:

- TP: True Positives (Correctly predicted Spam)
- TN: True Negatives (Correctly predicted Not Spam)
- FP: False Positives (Not Spam predicted as Spam)
- FN: False Negatives (Spam predicted as Not Spam)

2. Key Metrics (Calculated at Threshold = 0.50)

Accuracy
= (TP + TN) / Total
= (3,320 + 4,920) / 10,000
= 8,240 / 10,000
= 0.824 → 82.4%

Precision (Spam class)
= TP / (TP + FP)
= 3,320 / (3,320 + 1,080)
= 3,320 / 4,400
= 0.755 → 75.5%

Recall / Sensitivity (Spam class)
= TP / (TP + FN)
= 3,320 / (3,320 + 680)
= 3,320 / 4,000
= 0.830 → 83.0%

Specificity (Not Spam class)
= (TN / (TN + FP)
= 4,920 / (4,920 + 1,080)
= 4,920 / 6,000
= 0.820 → 82.0%

F1-Score (Spam class)
= 2 × (Precision × Recall) / (Precision + Recall)
= 2 × (0.755 × 0.830) / (0.755 + 0.830)
= 2 × 0.6266 / 1.585
= 0.790 → 79.0%

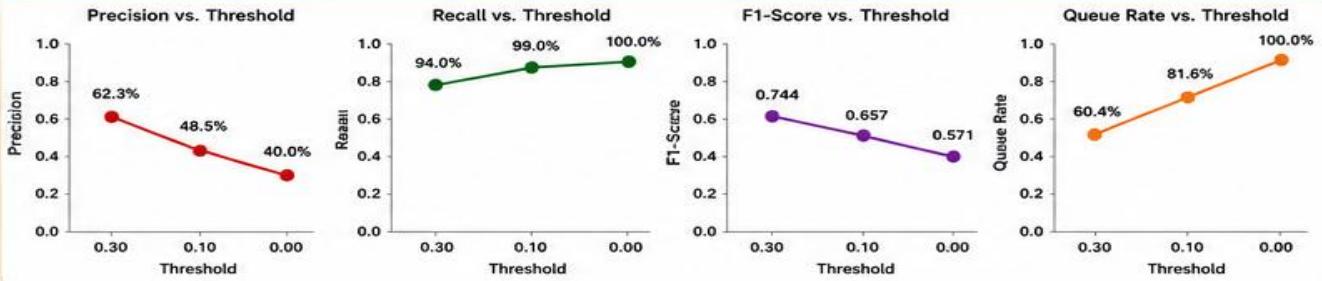
3. Discrimination Threshold Analysis

Threshold (Probability Cut-off)	TP (Spam Correctly Identified)	FP (Not Spam wrongly Identified)	Precision TP / (TP + FP)	Recall TP / (TP + FN)	F1-Score	Queue Rate (Flagged as Spam)
0.90	1,400	120	92.1%	35.0%	0.510	15.2%
0.70	2,480	420	85.5%	62.0%	0.712	29.0%
0.50	3,320	1,080	75.5%	83.0%	0.790	44.0%
0.30	3,760	2,280	62.3%	94.0%	0.744	60.4%
0.10	3,960	4,200	48.5%	99.0%	0.657	81.6%

High Threshold (Strict) → Low Threshold (Liberal)

As threshold decreases → Recall increases, Precision decreases, Queue rate increases.

4. Metrics vs. Threshold



5. Why Discrimination Thresholds Matter?

- Control the trade-off between catching more spam (Recall) and reducing false alarms (Precision).
- Reduce risk of missed threats (False Negatives) or user annoyance (False Positives).
- Impact operational cost: higher queue rate means more emails to review.
- Choose threshold based on business goals and domain constraints.

6. How to Choose the Right Threshold?

Business Goal	Preferred Threshold	Effect	Example Use Case
Minimize False Negatives (Catch more spam)	Low (e.g., 0.10 – 0.30)	Recall High ↑ Precision Low ↓	Security, fraud prevention, phishing emails
Balanced Trade-off	Medium (e.g., 0.40 – 0.60)	Precision & Recall Balanced	General purpose email filters
Minimize False Positives (Avoid wrong spam)	High (e.g., 0.70 – 0.90)	Precision High ↑ Recall Low ↓	Important business emails, customer communications

Key Takeaway: Discrimination thresholds make the decision boundary explicit and measurable.
The right threshold balances performance metrics with business impact.

Regression Model Visualization: Overview

- Regression diagnostics reveal whether model assumptions (linearity, homoscedasticity, normal residuals) hold.
- Key tools: residual plots, prediction error plots, alpha selection plots, Cook’s distance.

Residual Plots

- Plot residuals (actual − predicted) against predicted values or features.
- Random scatter around zero indicates a good fit; visible patterns (curves, funnels) indicate model misspecification or heteroscedasticity.
- A simple yet powerful explanatory tool: shows exactly where and how much the model errs.

Linear Regression – 10 Sample Example

We predict House Price (₹ Lakhs) using Area (sq.ft.)

Fitted Linear Regression Model:

$\hat{Y}$ (Predicted Price) = 2.484 + 0.0495 × Area

Model Performance Summary

R^2 = 0.971 Adjusted R^2 = 0.966

Modified R^2 = 0.962

RMSE = 0.523 Lakhs

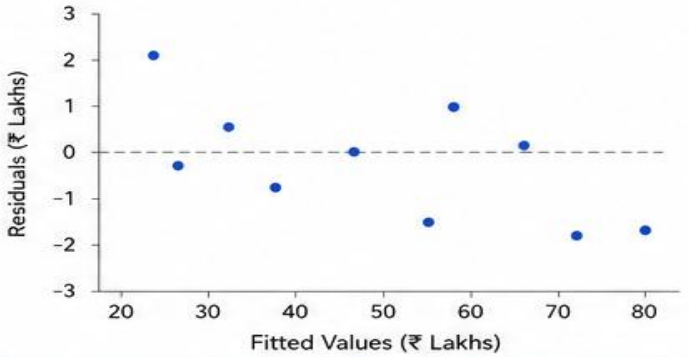
MAE = 0.470 Lakhs

1. Dataset with Predicted Values (10 Samples)

ID	Area (sq.ft.) (X)	Actual Price (₹ Lakhs) (Y)	Predicted Price (₹ Lakhs) ($\hat{Y}$)	Residual ($e = Y - \hat{Y}$)
1	500	25.0	27.234	-2.234
2	600	32.0	32.184	-0.184
3	700	36.0	37.134	-1.134
4	800	41.0	42.084	-1.084
5	900	47.0	47.034	-0.034
6	1000	52.0	51.984	0.016
7	1100	58.0	56.934	1.066
8	1200	62.0	61.884	0.116
9	1300	65.0	66.834	-1.834
10	1400	70.0	71.784	-1.784

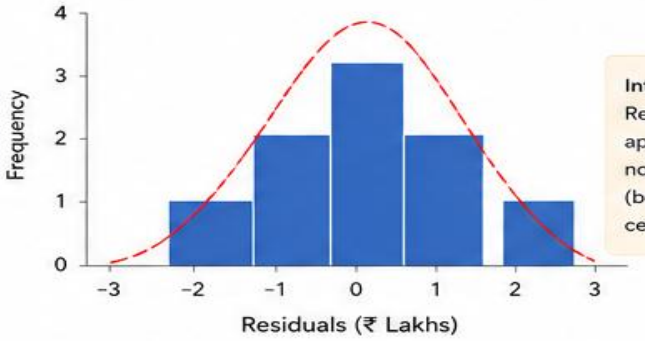
Note: Residual = Actual Price – Predicted Price

2. Residuals vs Fitted Values



Interpretation: Residuals are randomly scattered around zero with no clear pattern. Linearity and constant variance assumptions look reasonable.

3. Histogram of Residuals



Interpretation: Residuals are approximately normally distributed (bell-shaped) and centered around 0.

4. R^2 Calculation

Formula: $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$

Where:

- $SS_{res} = \sum (Y - \hat{Y})^2$ (Sum of Squared Residuals)
- $SS_{tot} = \sum (Y - \bar{Y})^2$ (Total Sum of Squares)

From calculations:

- SS_{res} = 6.194
- SS_{tot} = 213.100

$$R^2 = 1 - \frac{6.194}{213.100} = 1 - 0.02906 = 0.971$$

Interpretation: Model explains 97.1% of the variation in house prices.

5. Adjusted R^2 Calculation

Formula: $R^2_{adj} = 1 - (1 - R^2) \times \frac{n-1}{n-p-1}$

Where:

- R^2 = 0.971
- n = 10 (number of samples)
- p = 1 (number of predictors)

Calculation:

$$R^2_{adj} = 1 - (1 - 0.971) \times \frac{10-1}{10-1-1}$$

$$= 1 - (0.029) \times \frac{9}{8} = 0.966$$

Interpretation: Adjusted for number of predictors, adjusted R^2 is 0.966.

6. Modified R^2 Calculation

Formula: $R^2_{mod} = 1 - \frac{(1 - R^2)(n-1)}{(n-p)}$

Where:

- R^2 = 0.971
- n = 10 (number of samples)
- p = 1 (number of predictors)

Calculation:

$$R^2_{mod} = 1 - \frac{(1 - 0.971)(10-1)}{(10-1)}$$

$$= 1 - \frac{0.029 \times 9}{9} = 0.962$$

Interpretation: Modified R^2 is 0.962 (penalizes adding unnecessary predictors).

7. RMSE Calculation

Formula: $RMSE = \sqrt{\frac{SS_{res}}{n}}$

Where:

- SS_{res} = 6.194
- n = 10

Calculation:

$$RMSE = \sqrt{\frac{6.194}{10}} = \sqrt{0.6194} = 0.523$$

Interpretation: On average, model's predictions are off by ₹0.523 Lakhs.

8. MAE Calculation

Formula: $MAE = \frac{\sum |Y - \hat{Y}|}{n}$

Where:

- $\sum |Y - \hat{Y}|$ = 4.700
- n = 10

Calculation:

$$MAE = \frac{4.700}{10} = 0.470$$

Interpretation: On average, model's predictions are off by ₹0.470 Lakhs.

9. Summary of Model Performance

R^2 Score	Adjusted R^2	Modified R^2	RMSE (₹ Lakhs)	MAE (₹ Lakhs)
0.971	0.966	0.962	0.523	0.470

Overall Interpretation: The linear regression model fits the data very well. It explains 97.1% of the variation in house prices based on area and has low prediction errors (RMSE = 0.523, MAE = 0.470 Lakhs).

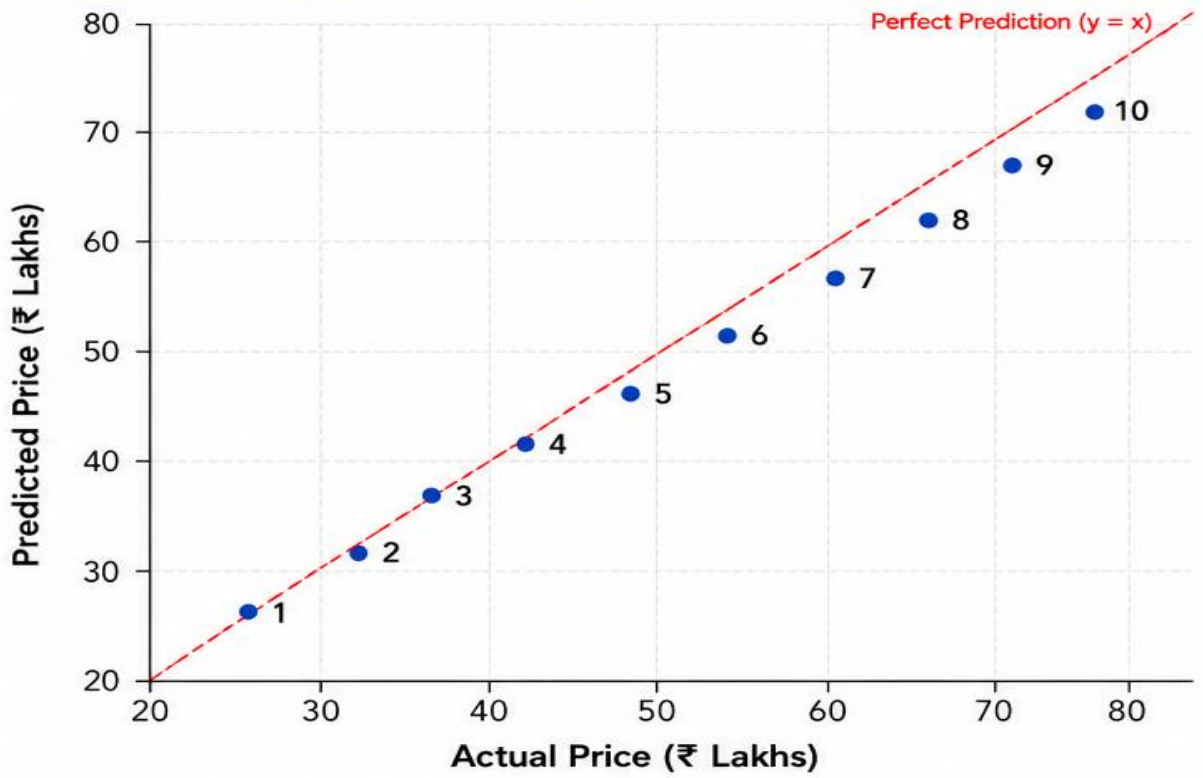
Prediction Error Plots

- Scatter plot of actual vs. predicted values, with a 45-degree reference line representing perfect prediction.
- Deviation from the diagonal reveals systematic over- or under-prediction in specific value ranges.
- Provides an intuitive, visual measure of model fit beyond aggregate metrics like R^2.

Actual vs. Predicted House Prices (10 Samples)

Linear Regression Model: $\hat{Y}$ **(Predicted Price)** $= 2.484 + 0.0495 \times$ **Area**

Scatter Plot: Actual vs. Predicted Prices



- Each point represents one sample.
- X-axis: Actual price (₹ Lakhs)
- Y-axis: Predicted price (₹ Lakhs)
- Points close to the 45° line indicate accurate predictions.

Data Used (10 Samples)

ID	Area (sq.ft.) (X)	Actual Price (₹ Lakhs) (Y)	Predicted Price (₹ Lakhs) ($\hat{Y}$)	Residual ($e = Y - \hat{Y}$)
1	500	25.0	27.234	−2.234
2	600	32.0	32.184	−0.184
3	700	36.0	37.134	−1.134
4	800	41.0	42.084	−1.084
5	900	47.0	47.034	−0.034
6	1000	52.0	51.984	0.016
7	1100	58.0	56.934	1.066
8	1200	62.0	61.884	0.116
9	1300	65.0	66.834	−1.834
10	1400	70.0	71.784	−1.784

Conclusion: The scatter plot shows that all actual vs. predicted prices lie very close to the 45° line, indicating a very good fit of the linear regression model.

Alpha Selection Plots (feature reduction)

- Used for regularized regression (Ridge, Lasso) to visualize how coefficients and error change with the regularization parameter α.
- Helps select an α that balances bias and variance while maintaining interpretable, non-degenerate coefficients.
- Demonstrates explicitly how regularization shrinks or eliminates feature coefficients – directly relevant to interpretability.

Alpha Selection in Lasso Regression (House Price Prediction Example)

Dataset Used (10 Houses)

House ID	Area (sq.ft.)	Bedrooms	Age (years)	Distance to CBD (km)	Parking (Yes=1, No=0)	Actual Price (₹ Lakhs)
H1	800	2	12	8	1	45
H2	900	2	10	7	1	52
H3	980	3	9	6	1	58
H4	1050	3	8	5	1	63
H5	1120	3	7	5	0	70
H6	1200	4	6	4	1	75
H7	1280	4	5	4	1	81
H8	1380	4	4	3	1	89
H9	1480	5	3	2	1	94
H10	1600	5	2	2	1	102

5 Predictors (Features): Area, Bedrooms, Age, Distance to CBD, Parking
Target (Response): Actual Price (₹ Lakhs)

Linear Regression (Ordinary Least Squares)

Linear regression estimates coefficients by minimizing the Sum of Squared Errors (SSE):

$$\hat{\beta}_{OLS} = \arg\min_{\beta_0,\beta} \sum_{i=1}^{n} (y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j)^2$$

Where,

- y_i = Actual price of house i
- x_{ij} = Value of predictor j for house i
- β_0 = Intercept, β_j = Coefficient for predictor j
- n = Number of observations (houses), p = Number of predictors (5)

Lasso Regression (L1 Regularization)

Lasso regression estimates coefficients by minimizing the following objective function:

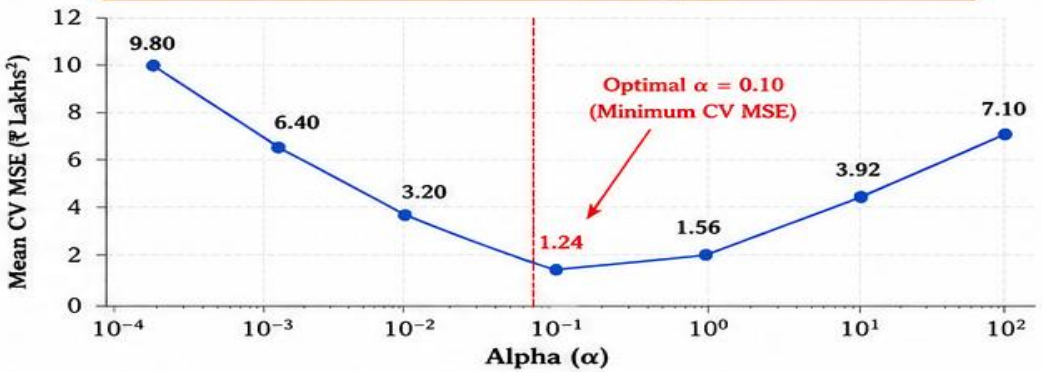
$$\hat{\beta}(\alpha) = \arg\min_{\beta_0,\beta} \left[\underbrace{\sum_{i=1}^{n} (y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j)^2}_{\text{Least Squares Loss}} + \underbrace{\lambda \sum_{j=1}^{p} |\beta_j|}_{\text{L1 Penalty}} \right]$$

Where, λ (alpha, α) controls the penalty strength.

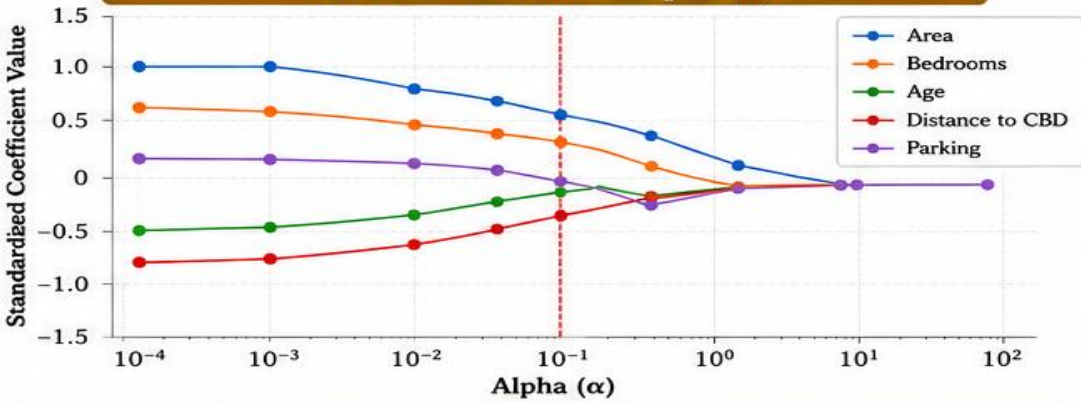
Higher α → stronger penalty → more coefficients shrink to zero.

1. **Dataset** — 10 Houses, 6 Features (5 Predictors + Target)
2. **Train Models** — Fit Linear Regression and Lasso Regression
3. **Try Multiple Alpha (α) Values** — Range of α from 10^{-4} to 10^{2} (Log Scale)
4. **Cross Validation** — 5-Fold CV to compute Mean Squared Error
5. **Select Optimal α** — Alpha with Minimum CV Error
6. **Feature Selection** — Coefficients Shrinked, Irrelevant Features Eliminated
7. **Final Explainable Model** — Model with Important Features Only

Cross Validation Error vs Alpha (α)



Coefficient Paths vs Alpha (α)



Model Comparison at Optimal α = 0.10

Model	No. of Non-zero Coefficients	Training MSE (₹ Lakhs²)	5-Fold CV MSE (₹ Lakhs²)	R^2 (CV)
Linear Regression (OLS)	5	0.62	1.46	0.91
Lasso Regression (α = 0.10)	3	0.78	**1.24**	**0.93**

Lasso achieves lower CV MSE and better generalization with fewer features.

Selected Alpha (α)
0.10

Minimum CV MSE
1.24 (₹ Lakhs²)

Important Features Retained (Non-zero Coefficients)

- Area
- Bedrooms
- Distance to CBD

Removed Features (Zero Coefficients at α = 0.10)

- Parking
- Age

Final Lasso Model (at α = 0.10)

$$\hat{y} = 2.35 + 0.0042\,(\text{Area}) + 6.10\,(\text{Bedrooms}) - 4.85\,(\text{Distance to CBD})$$

(Age and Parking coefficients = 0)

Key Observation

As α increases, the L1 penalty shrinks less important coefficients toward zero. The CV Error plot helps identify the optimal α that minimizes prediction error, while the Coefficient Path plot shows which features are retained. Here, α = 0.10 gives the best trade-off between accuracy and interpretability.

Cook’s Distance (outlier detection)

- Measures the influence of each individual observation on the fitted regression model.
- High Cook’s distance values flag potential outliers or high-leverage points that disproportionately affect model coefficients.
- Important for explainability: an unexplainable prediction may simply be driven by one anomalous training point.

Cook's Distance – Calculation Example (with Leverage Calculation)

Example: House Price Prediction using Multiple Linear Regression
Predictors: Area (sq.ft.), Bedrooms, Age (years) | Target: Price (₹ Lakhs)

1. Dataset with Actual vs Predicted Prices (10 Houses)

House ID	Actual Price y_i (₹ Lakhs)	Predicted Price $\hat{y}_i$ (₹ Lakhs)	Residual $e_i = y_i - \hat{y}_i$	Residual² e_i^2
1	45	46	−1	1
2	52	50	2	4
3	58	60	−2	4
4	63	61	2	4
5	70	72	−2	4
6	75	74	1	1
7	81	83	−2	4
8	89	87	2	4
9	94	96	−2	4
10	**102**	**110**	**−8**	**64**

$$\text{RSS} = \sum e_i^2 = 94$$

Step 2: $(X^T X)^{-1}$ (Computed from all 10 observations)

$$(X^T X)^{-1} = \begin{bmatrix} 0.034269 & -0.000175 & -0.009714 & 0.001868 \\ -0.000175 & 0.000002 & 0.000002 & -0.000000 \\ -0.009714 & 0.000002 & 0.268073 & -0.016271 \\ 0.001868 & -0.000000 & -0.016271 & 0.001204 \end{bmatrix}$$

($n = 10$ observations, $p = 4$ parameters: intercept + 3 predictors)

2. Leverage Calculation for One Point (House 10)

We will compute leverage h_{10} for House 10.

Formula:

$$h_{ii} = x_i^T (X^T X)^{-1} x_i$$

Where,

- x_i = row vector of predictors for observation i (including intercept)
- X = design matrix ($n \times p$)
- h_{ii} = leverage of observation i

Step 1: Design matrix X ($n = 10$, $p = 4$)

Obs	Intercept	Area (A)	Bedrooms (B)	Age (G)
1	1	800	2	12
2	1	900	2	10
3	1	980	3	9
...	...	...	...	...
10	**1**	**1600**	**5**	**2**

Step 3: Predictor vector for House 10

$$x_{10} = \begin{bmatrix} 1 & 1600 & 5 & 2 \end{bmatrix}$$

Step 4: Compute leverage h_{10}

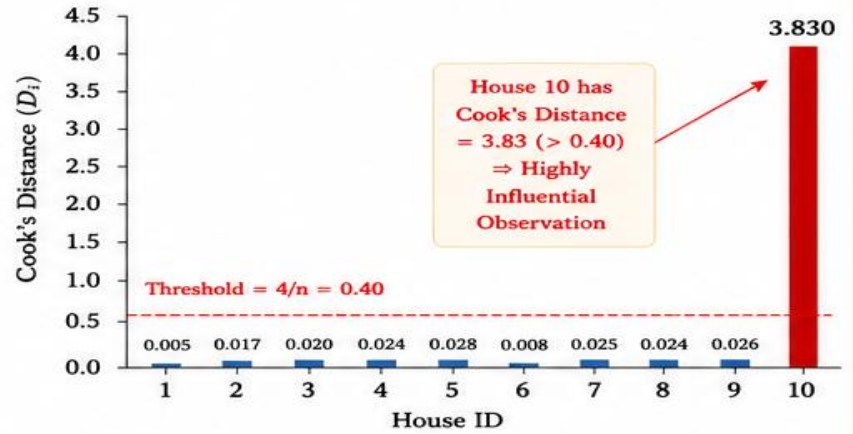
$$h_{10} = x_{10}^T (X^T X)^{-1} x_{10}$$

$$= \begin{bmatrix} 1 & 1600 & 5 & 2 \end{bmatrix} \begin{bmatrix} 0.034269 & -0.000175 & -0.009714 & 0.001868 \\ -0.000175 & 0.000002 & 0.000002 & -0.000000 \\ -0.009714 & 0.000002 & 0.268073 & -0.016271 \\ 0.001868 & -0.000000 & -0.016271 & 0.001204 \end{bmatrix} \begin{bmatrix} 1 \\ 1600 \\ 5 \\ 2 \end{bmatrix}$$

$$= \mathbf{0.6003}$$

Thus, leverage of House 10 (h_{10}) = **0.6003**

3. Cook's Distance for Each Observation



4. Cook's Distance Results

House ID	Residual (e_i)	Residual² (e_i^2)	Leverage (h_{ii})	Cook's Distance (D_i)	Interpretation
1	−1	1	0.1502	0.005	Not Influential
2	2	4	0.1806	0.017	Not Influential
3	−2	4	0.1975	0.020	Not Influential
4	2	4	0.2177	0.024	Not Influential
5	−2	4	0.2461	0.028	Not Influential
6	1	1	0.2373	0.008	Not Influential
7	−2	4	0.2284	0.025	Not Influential
8	2	4	0.2171	0.024	Not Influential
9	−2	4	0.2386	0.026	Not Influential
10	**−8**	**64**	**0.6003**	**3.830**	**Highly Influential**

5. Interpretation

- Cook's Distance identifies observations that have a strong influence on the regression model.
- Threshold $= \frac{4}{n} = \frac{4}{10} = 0.40$
- Houses 1–9 have Cook's Distance < 0.40 → Normal observations.
- House 10 has large residual (−8) and high leverage (0.6003).
- Its Cook's Distance (3.83) is much greater than the threshold (0.40).
- Therefore, House 10 is a highly influential point and should be investigated.

6. Key Takeaway

Cook's Distance combines two aspects:

1. How unusual the predictor values are (Leverage)
2. How large the prediction error is (Residual)

A point with both high leverage and large residual can strongly affect the fitted regression model.

Cook's Distance Formula

$$D_i = \frac{e_i^2}{p \times MSE} \times \frac{h_{ii}}{(1 - h_{ii})^2}$$

Where,

- e_i = Residual for observation i
- h_{ii} = Leverage of observation i
- p = Number of parameters (including intercept) = 4
- MSE = Mean Squared Error $= \frac{RSS}{n - p} = \frac{94}{10 - 4} = 15.67$

Note: If an influential observation is due to data entry error, it may be corrected or removed. If it is a valid extreme case, it should be kept but reported.

Q-Q PLOT (NORMALITY CHECK OF RESIDUALS)

Example: House Price Prediction using Multiple Linear Regression
Predictors: Area (sq.ft.), Bedrooms, Age (years) | Target: Price (₹ Lakhs)

1. Residuals from Regression Model

House ID	Actual Price (₹ Lakhs)	Predicted Price (₹ Lakhs)	Residual $e_i = y_i - \hat{y}_i$
1	45.0	44.8	0.2
2	52.0	50.6	1.4
3	58.5	59.7	−1.2
4	63.2	61.8	1.4
5	70.0	72.1	−2.1
6	75.5	73.9	1.6
7	81.0	82.9	−1.9
8	89.0	88.4	0.6
9	94.8	95.6	−0.8
10	102.0	111.3	−9.3

Residual Sum of Squares (RSS) = $\sum e_i^2 = 106.59$
Mean Squared Error (MSE) = $106.59 / (10 - 4) = 17.77$
Root MSE (RMSE) = 4.21

2. Sort Residuals (Ascending Order)

Rank (i)	Ordered Residual $e_{(i)}$
1	−9.3
2	−2.1
3	−1.9
4	−1.2
5	−0.8
6	0.2
7	0.6
8	1.4
9	1.4
10	1.6

3. Calculate Plotting Positions

$$P_i = \frac{i - 0.5}{n}, \quad n = 10$$

Rank (i)	P_i
1	0.05
2	0.15
3	0.25
4	0.35
6	0.55
7	0.65
8	0.75
9	0.85
10	0.95

4. Convert P_i to Standard Normal Quantiles (Z_i)

P_i	Expected Z_i (Quantile)
0.05	−1.645
0.15	−1.036
0.25	−0.674
0.35	−0.385
0.45	−0.126
0.55	0.126
0.65	0.385
0.75	0.674
0.85	1.036
0.95	1.645

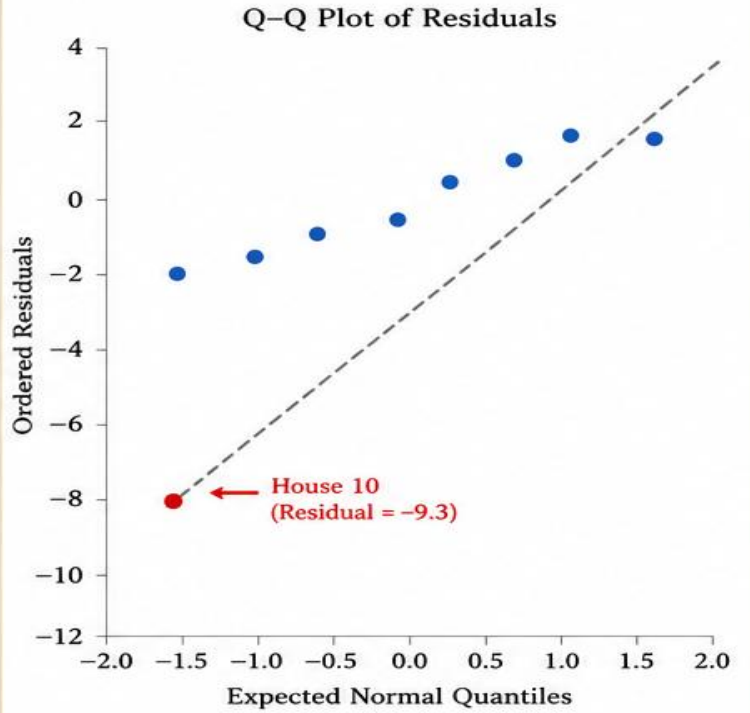
5. Data Used for Q–Q Plot

Rank (i)	P_i	Expected Z_i (Quantile)	Ordered Residual $e_{(i)}$
1	0.05	−1.645	−9.3
2	0.15	−1.036	−2.1
3	0.25	−0.674	−1.9
4	0.35	−0.385	−1.2
5	0.45	−0.126	−0.8
6	0.55	0.126	0.2
7	0.65	0.385	0.6
8	0.75	0.674	1.4
9	0.85	1.036	1.4
10	0.95	1.645	1.6

How to Construct Q–Q Plot

- Compute residuals from the model.
- Sort residuals in ascending order.
- Compute plotting positions P_i.
- Convert P_i to expected normal quantiles (Z_i).
- Plot ordered residuals (Y-axis) against expected quantiles (X-axis).
- Add 45° reference line.

6. Q–Q Plot



7. Interpretation

- Most of the points lie close to the 45° reference line, indicating that the residuals are approximately normally distributed.
- House 10 lies far below the line due to a very large negative residual (−9.3).
- This observation is a potential outlier and has high influence on the model (as seen in Cook's Distance).
- **Investigate this point before finalizing the regression model.**

Normality of residuals is one of the key assumptions of linear regression.
Q–Q plot provides a visual assessment of this assumption.

Clustering Model Visualization: Overview

- Unsupervised models lack ground-truth labels, making visualization essential for validating and explaining cluster structure.
- Key tools: elbow method, silhouette coefficient visualizer, intercluster distance maps.

Elbow Method

- Plots within-cluster sum of squares (inertia) against the number of clusters k.
- The “elbow” point – where adding more clusters yields diminishing returns – suggests a good choice of k.
- A simple, visual, and interpretable heuristic for model selection in clustering.

Elbow Method for Choosing the Optimal Number of Clusters (k)

Plots within-cluster sum of squares (inertia) against the number of clusters k.

What is Inertia (Within-Cluster Sum of Squares)?

Inertia measures how compact the clusters are.
It is the sum of squared distances between each point and the centroid of its cluster.

$$\text{Inertia} = \sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu_i \|^2$$

- k = Number of clusters
- C_i = Points in cluster i
- μ_i = Centroid of cluster i
- $\| x - \mu_i \|$ = Euclidean distance between point x and centroid μ_i

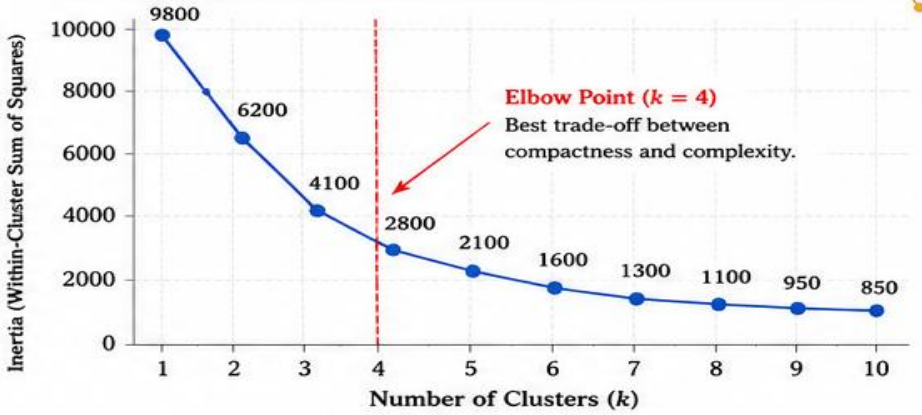
How the Elbow Method Works

1. Run k-means clustering for a range of k values (e.g., 1 to 10).
2. For each k, compute the inertia (within-cluster sum of squares).
3. Plot inertia (y-axis) against the number of clusters k (x-axis).
4. Identify the "elbow" point — where the rate of decrease in inertia sharply changes and starts to level off. The k at this point is a good choice.

Key Idea

Beyond the elbow, adding more clusters gives only small improvements in inertia (diminishing returns) but increases model complexity.

Elbow Plot: Inertia vs Number of Clusters (k)



Interpretation

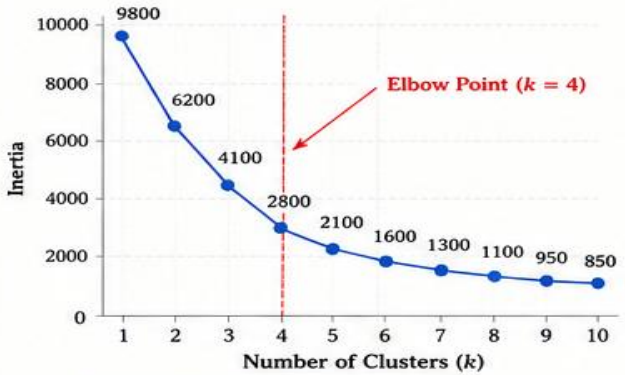
From $k = 1$ to $k = 4$, inertia decreases rapidly (big improvement).
From $k = 4$ onward, the decrease is gradual (small improvement).
Therefore, $k = 4$ is chosen as the **optimal number of clusters.**

Example: Customer Segmentation Using K-Means Clustering

Sample Dataset (2 Features)

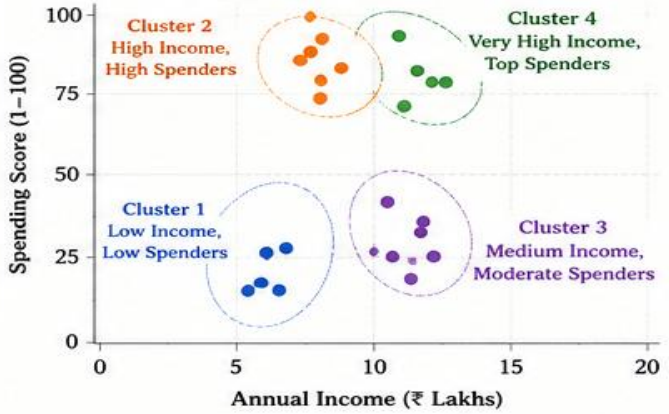
Customer ID	Annual Income (₹ Lakhs)	Spending Score (1–100)
C1	3.4	20
C2	4.1	25
C3	3.9	18
C4	8.2	75
C5	9.1	80
C6	8.7	70
C7	15.2	90
C8	16.5	95
C9	14.8	88
C10	4.5	22
...	...	...

Step 1: Elbow Plot



Result: Optimal $k = 4$

Step 2: Clusters Formed ($k = 4$)



Step 3: Interpretation

Cluster	Description	Strategy
1 (Blue)	Low Income, Low Spenders	Budget offers, awareness campaigns
2 (Orange)	High Income, High Spenders	Loyalty programs, Premium offers
3 (Purple)	Medium Income, Moderate Spenders	Upsell opportunities, Personalized offers
4 (Green)	Very High Income, Top Spenders	VIP services, Exclusive benefits

Why It Works Well?

Simple & Visual: Easy to understand and communicate.

Interpretable: The elbow clearly shows diminishing returns.

Effective Trade-off: Balances cluster compactness and model simplicity.

Summary

- Use inertia vs k plot.
- Find the elbow.
- Choose k at the elbow for the best clustering solution.

Silhouette Coefficient Visualizer

- Computes a silhouette score per sample, measuring how similar it is to its own cluster vs. other clusters.
- Visualized as per-cluster silhouette plots, revealing cluster cohesion and separation quality.
- Negative or near-zero silhouette values indicate poorly assigned or ambiguous points – useful for explaining clustering quality.

Silhouette Coefficient Calculater for a Simple Clustering Example (With Calculations)

Measures how well each point lies within its cluster compared to other clusters. Range: -1 to 1 (Higher is better)

1. What is Silhouette Coefficient?

It measures how similar an object is to its own cluster (cohesion) compared to other clusters (separation).

- **Close to 1:** Well clustered
- **Around 0:** On the boundary
- **Close to -1:** Probably in the wrong cluster

2. Formula & Definitions

For each point i:

$a(i)$ = average distance from i to all other points in the same cluster (0 if cluster has only one point)

$b(i)$ = minimum average distance from i to all points in any other cluster

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad \text{where} \quad -1 \le s(i) \le 1$$

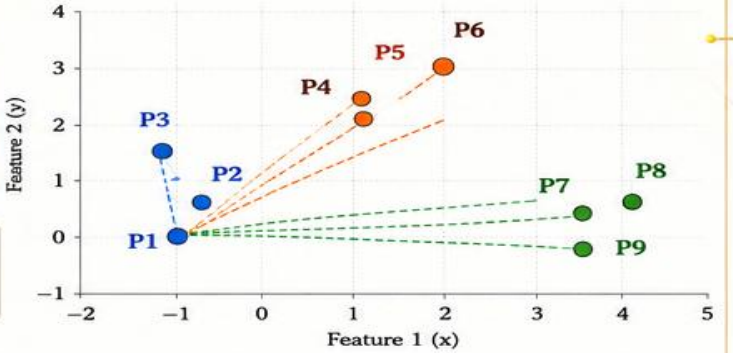
Overall Silhouette Score = average of $s(i)$ for all points

3. Visualization: Distances for Point P1 (to compute a and b)

For Point P1 (in C1)

$a(1)$: Avg. distance to points in C1 (P2, P3)

$b(1)$: Min. avg. distance to points in other clusters (C2 or C3)



Compute $a(1)$

Distances: $d(P_1, P_2) = 0.71$, $d(P_1, P_3) = 1.00$

$$\Rightarrow a(1) = \frac{0.71 + 1.00}{2} = 0.855 \Rightarrow 0.86$$

Compute $b(1)$

Avg. dist. to C2: 3.18, Avg. dist. to C3: 3.00

$$\Rightarrow b(1) = \min(3.18, 3.00) = 3.00$$

Silhouette of P1

$$s(1) = \frac{b(1) - a(1)}{\max(a(i), b(i))} = \frac{3.00 - 0.86}{\max(0.86, 3.00)} = \frac{2.14}{3.00} = 0.71$$

4. Dataset (9 Points, 2 Features)

Point	Feature 1 (x)	Feature 2 (y)
P1	−1.0	0.0
P2	−0.5	0.5
P3	−1.0	1.0
P4	2.0	2.5
P5	2.5	3.0
P6	1.5	2.0
P7	3.0	0.5
P8	3.5	1.0
P9	3.0	0.0

We compute silhouette coefficient for $k = 3$ clusters.

Cluster 1 (C1): P1, P2, P3

Cluster 2 (C2): P4, P5, P6

Cluster 3 (C3): P7, P8, P9

Distance used: Euclidean distance

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

5. Silhouette Score Calculation for $k = 3$ Clusters (Detailed)

Point	Cluster Label	$a(i)$ (Avg. dist. to same cluster)		$b(i)$ (Min. avg. dist. to other clusters)			$s(i) = b(i) - a(i) / \max(a(i), b(i))$	Interpretation
		Distances to same cluster points	$a(i)$	Avg. dist. to Cluster 2	Avg. dist. to Cluster 3	$b(i)$ = min		
P1	C1	to P2 = 0.71, P3 = 1.00	0.86	3.18	3.00	3.00	0.71	Well inside
P2	C1	to P1 = 0.71, P3 = 0.71	0.71	2.86	2.71	2.71	0.74	Well inside
P3	C1	to P1 = 1.00, P2 = 0.71	0.86	2.86	2.50	2.50	0.66	Well inside
P4	C2	to P5 = 0.71, P6 = 1.12	0.91	2.86	2.06	2.06	0.56	Well inside
P5	C2	to P4 = 0.71, P6 = 1.12	0.91	3.16	2.06	2.06	0.56	Well inside
P6	C2	to P4 = 1.12, P5 = 1.12	1.12	2.71	1.64	1.64	0.32	On boundary
P7	C3	to P8 = 0.71, P9 = 0.50	0.61	3.00	2.36	2.36	0.74	Well inside
P8	C3	to P7 = 0.71, P9 = 0.71	0.71	3.16	2.19	2.19	0.68	Well inside
P9	C3	to P7 = 0.50, P8 = 0.71	0.61	3.04	2.19	2.19	0.72	Well inside
Average Silhouette Score for $k = 3$ $\Rightarrow$							**0.72**	

6. Average Silhouette Score for Different k

k (No. of Clusters)	Average Silhouette Score	Interpretation
2	0.45	Fair (clusters not well separated)
3	**0.72**	Best (well separated clusters)
4	0.54	Moderate (over-segmentation)
5	0.38	Poor (too many small clusters)

7. Silhouette Score Scale

😊	0.71 – 1.00	Excellent
🙂	0.51 – 0.70	Good
😐	0.26 – 0.50	Fair
☹️	0.01 – 0.25	Poor
😞	−1.00 – 0.00	Very Poor

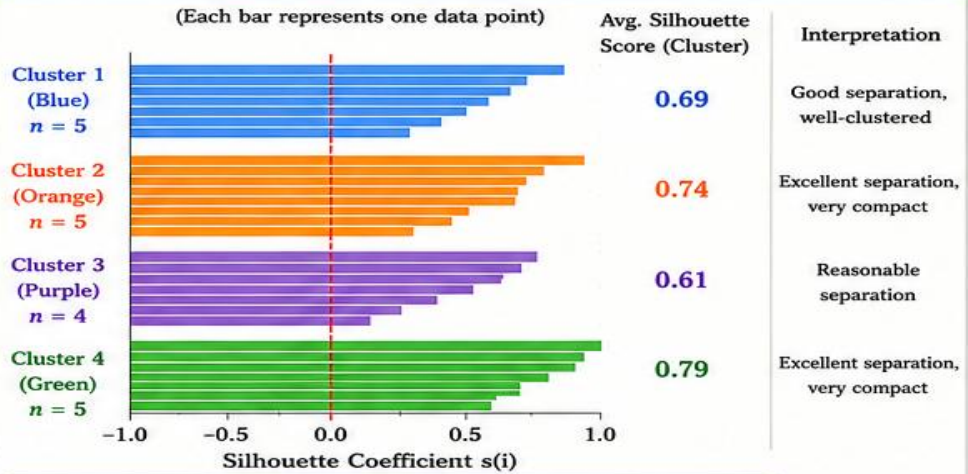
Silhouette Coefficient Visualizer for Selecting the Optimal Number of Clusters (k)

Example: Customer Segmentation Using K-Means Clustering (Same Dataset as Elbow Example)

Sample Dataset (2 Features)

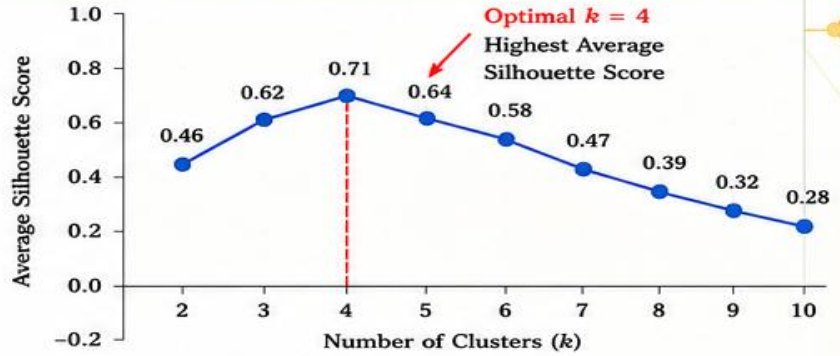
Customer ID	Annual Income (₹ Lakhs)	Spending Score (1–100)
C1	3.4	20
C2	3.9	75
C3	3.9	18
C4	8.2	75
C5	9.1	80
C6	8.7	70
C7	15.2	90
C8	16.5	95
C9	14.8	88
C10	4.5	22
...	...	...

Silhouette Plot for $k = 4$



Overall Average Silhouette Score ($k = 4$) = 0.71

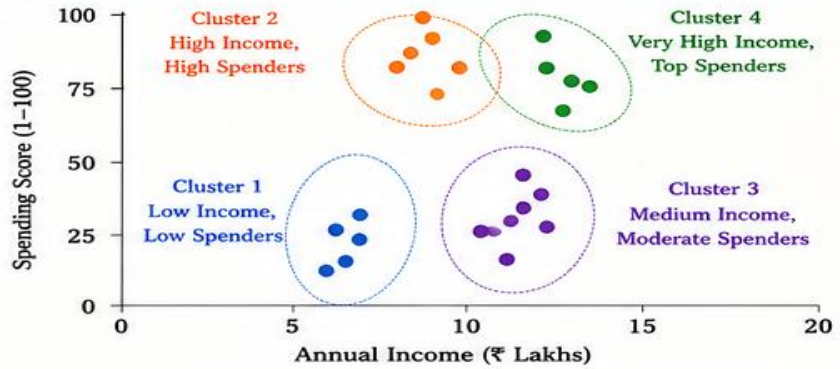
Average Silhouette Score vs Number of Clusters (k)



Interpretation

The average silhouette score is highest at $k = 4$ (0.71), indicating well-separated and compact clusters. Therefore, $k = 4$ is the optimal number of clusters.

Clusters Formed ($k = 4$)



Silhouette Score by k

k	Average Silhouette Score	Interpretation
2	0.46	Weak structure
3	0.62	Reasonable
4	**0.71**	**Strong structure (Best)**
5	0.64	Reasonable
6	0.58	Weak
7	0.47	Weak
8	0.39	Weak
9	0.32	Very Weak
10	0.28	Very Weak

Why Silhouette Coefficient?

- Considers both cluster compactness and separation.
- Works for any clustering algorithm (e.g., K-Means, Hierarchical, DBSCAN*).

*For DBSCAN, silhouette is computed for core points.

How to Choose k?

- Look for the highest average silhouette score.
- If multiple k values are close, prefer smaller k (simpler model).

What Does It Tell Us?

- Helps avoid under-clustering (too few clusters).
- Helps avoid over-clustering (too many clusters).
- Gives a balance between simplicity and accuracy.

Summary for This Example

Elbow Method suggested: $k = 4$

Silhouette Method suggested: $k = 4$

Final Optimal Number of Clusters: $k = 4$

Intercluster Distance Maps

- Project cluster centers into 2D space (e.g., via MDS) with circle size proportional to cluster membership.
- Visually communicates how well-separated clusters are, and which clusters are most similar to each other.
- Provides an intuitive global explanation of the overall clustering structure to non-technical stakeholders.

Intercluster Distance Maps for Understanding Cluster Separation

Example: Customer Segmentation Using K-Means Clustering ($k = 4$)

1. What are Intercluster Distances?

Intercluster distance measures the distance between the centroids (centers) of different clusters. It helps us understand how well-separated the clusters are.

For clusters C_i and C_j with centroids μ_i and μ_j:

$$D_{ij} = \|\mu_i - \mu_j\|_2 = \sqrt{\sum_{p=1}^{d} (\mu_{ip} - \mu_{jp})^2}$$

Where:

- d = Number of features (here, 2: Annual Income, Spending Score)
- D_{ij} = Euclidean distance between cluster i and j

2. How It Works

1. Run K-Means clustering and obtain cluster centroids.
2. Compute pairwise distances between all centroid pairs.
3. Visualize the distance matrix as a heatmap and/or network.
4. Use the map to assess cluster separation (larger distance = better separation).

Key Idea

Well-separated clusters have larger intercluster distances (warmer colors / thicker edges).

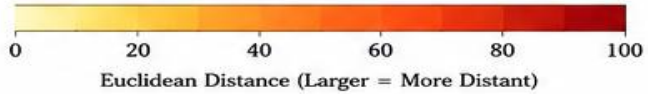
3. Cluster Centroids ($k = 4$)

Cluster	Annual Income (₹ Lakhs)	Spending Score (1–100)	Size (n)
1 (Blue)	4.22	20.00	5
2 (Orange)	8.75	72.50	6
3 (Purple)	13.10	56.00	4
4 (Green)	16.60	90.00	5

Centroid = Mean position of all points in the cluster (this is the "center" of the cluster in feature space).

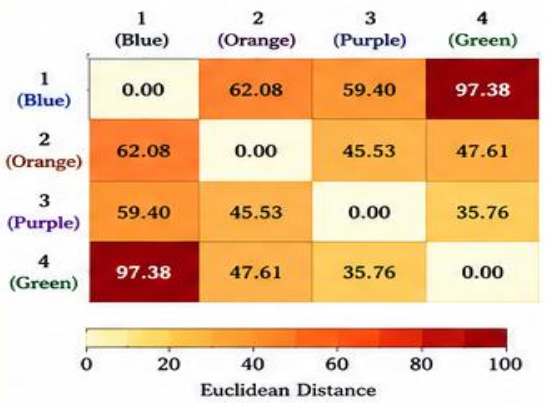
4. Intercluster Distance Matrix (Euclidean Distance)

From \ To	1 (Blue)	2 (Orange)	3 (Purple)	4 (Green)
1 (Blue)	0.00	62.08	59.40	97.38
2 (Orange)	62.08	0.00	45.53	47.61
3 (Purple)	59.40	45.53	0.00	35.76
4 (Green)	97.38	47.61	35.76	0.00

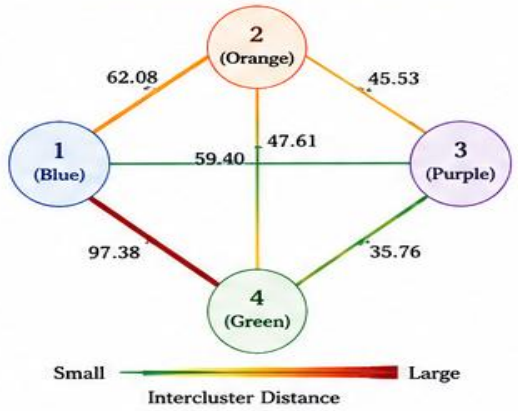


5. Intercluster Distance Maps (Visualizations)

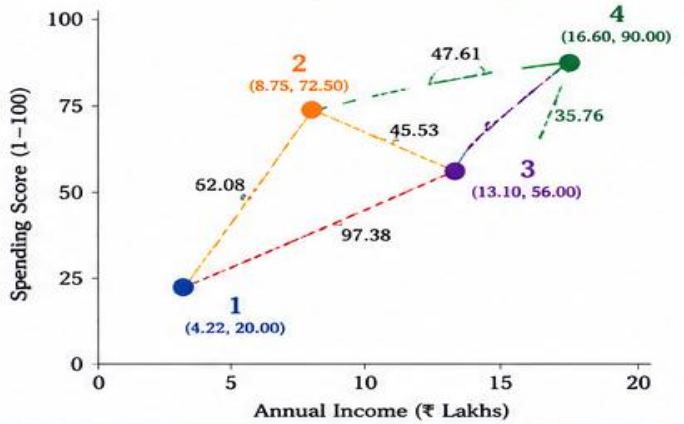
A. Distance Heatmap



B. Intercluster Distance Network



C. Centroid Map in 2D Feature Space



6. Interpretation

- Cluster 4 (Green) is far from Cluster 1 (Blue) (distance = 97.38) → Very distinct segments.
- Cluster 1 (Blue) is also relatively far from Clusters 2 and 3 (59.40 to 62.08).
- Clusters 2 (Orange) and 3 (Purple) are moderately separated (45.53).
- Clusters 3 (Purple) and 4 (Green) are the closest pair (35.76) but still reasonably apart.

Rule of Thumb

Larger intercluster distances indicate better separation and lower overlap between clusters.

7. Sample Dataset (2 Features)

Customer ID	Annual Income (₹ Lakhs)	Spending Score (1–100)	Cluster (Label)
C1	3.4	20	1 (Blue)
C2	4.1	25	1 (Blue)
C3	8.2	75	2 (Orange)
C4	9.1	80	2 (Orange)
C5	15.2	90	4 (Green)
...	...	...	...

8. Cluster Sizes

Cluster	Size (n)	Percentage (%)
1 (Blue)	5	20.8%
2 (Orange)	6	25.0%
3 (Purple)	4	16.7%
4 (Green)	5	20.8%
Total	20	100%

9. Summary Statistics

Min Intercluster Distance	35.76 (Between 3 & 4)
Max Intercluster Distance	97.38 (Between 1 & 4)
Average Intercluster Distance	57.29
Overall Separation	★★★★☆ (Good)

10. Key Takeaways

- Intercluster distance maps provide a clear visual and numerical summary of how well clusters are separated.
- They help validate the quality of clustering beyond inertia and silhouette scores.
- Use together with Silhouette Score and Elbow Method for robust model selection.

In This Example

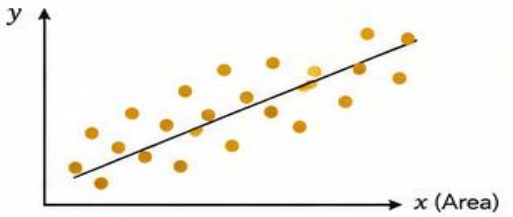
Clusters are well-separated overall. Cluster 4 (Green) and Cluster 1 (Blue) are the most distinct customer segments.

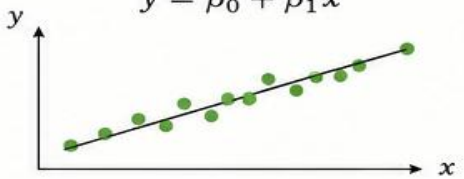
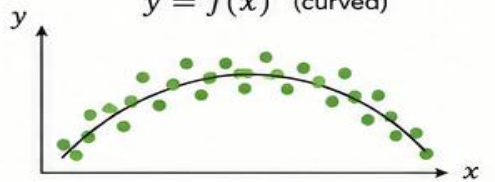
Good segmentation achieved!

Interpretable Machine Learning Properties

- Local or Global
- Linearity
- Monotonicity
- Feature Interactions
- Best-suited Complexity

Key Characteristics of Interpretable ML Models

Characteristic	What it means	Explanation / Example	Typical Models / Methods
Local or Global	• **Local:** explains a single prediction (instance level) • **Global:** explains the model behavior over the entire data space	**LOCAL** (Why this prediction?)  Explains this specific prediction (one instance at a time). **GLOBAL** (How does model behave overall?)  Explains the overall behavior of the model across the entire data space.	**Local:** • LIME • SHAP • Counterfactuals **Global:** • Linear/Logistic Regression (coefficients) • PDP (Partial Dependence Plots) • ALE (Accumulated Local Effects)

Characteristic	What it means	Explanation / Example	Typical Models / Methods
Linearity	• **Linear:** straight-line relationship between features and target • **Non-linear:** curved or complex relationship between features and target	**LINEAR RELATIONSHIP** $y = \beta_0 + \beta_1 x$  Example: House price vs Area Change in x results in a constant rate of change in y. **NON-LINEAR RELATIONSHIP** $y = f(x)$ (curved)  Example: Solar GHI vs Time of Day Change in x results in a varying rate of change in y.	**Linear Models:** • Linear Regression • Logistic Regression • Linear SVM **Non-linear Models:** • Decision Tree • Random Forest • XGBoost • Neural Networks • SVR (RBF Kernel)

Key Takeaway

Local vs Global:
Local explains one prediction,
Global explains the entire model behavior.

Linear vs Non-linear:
Choose model based on the nature of relationship in data; non-linear models capture complex patterns better.

Key Characteristics of Interpretable ML Models

Monotonicity

Does the relationship between the feature and the target go in the same direction over the entire feature domain?

Monotonically Increasing

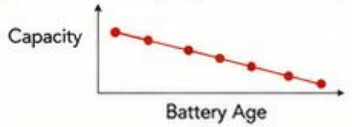
Income vs Loan Approval Probability



Higher income always increases approval probability.

Monotonically Decreasing

Battery Age vs Remaining Capacity



As battery age increases, capacity always decreases.

Non-monotonic Relationship

Solar Irradiance vs Time of Day



Rises in the morning, peaks at noon, then falls in the evening.

Why it matters: Many real-world problems require monotonic behavior based on domain knowledge. (Some algorithms allow monotonic constraints.)

Feature Interactions

Some models capture interactions between features while some assume independence. Interactions can increase quality but also increase complexity.

Without Interaction (Additive)

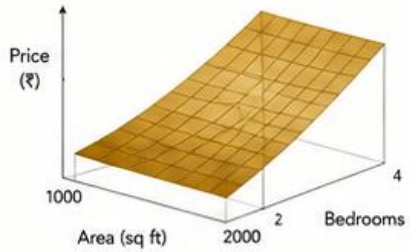
Price = β_1 Area + β_2 Bedrooms

Area (sq ft)	Bedrooms	Price (₹)
1000	2	40 L
1000	4	60 L
2000	2	90 L
2000	4	1.5 Cr

Each feature contributes independently.

With Interaction

Price = β_1 Area + β_2 Bedrooms + β_3 (Area × Bedrooms)



Effect of one feature depends on another (e.g., large house with more bedrooms → higher price).

Models like Trees, Random Forests, and Neural Networks automatically capture feature interactions.

Best-suited Complexity

Based on the hypothesis space of the model, what kinds of problem complexity is the algorithm best suited for?

Model	Hypothesis Space	Complexity	Interpretability
Linear Regression	Straight line	Low	★★★★☆
Logistic Regression	Linear separation	Low	★★★½☆
Decision Tree	Piecewise constant	Medium	★★★½☆
Random Forest	Ensemble of trees	High	★★☆☆☆
XGBoost	Boosted trees	High	★½☆☆☆
Neural Network	Can approximate any function	Very High	★☆☆☆☆

Lower Complexity (Higher Interpretability) → Higher Complexity (Higher Accuracy)

Choose the simplest model that achieves good performance. More complexity → less interpretability.

Key Takeaway

Monotonicity: Ensures feature influence remains in one direction (e.g., income ↑ → approval ↑).

Feature Interactions: Improve predictive power by capturing combined effects (e.g., area × bedrooms).

Best-suited Complexity: Match model complexity with problem complexity while balancing accuracy and interpretability.

Traditional Interpretable Algorithms: Overview

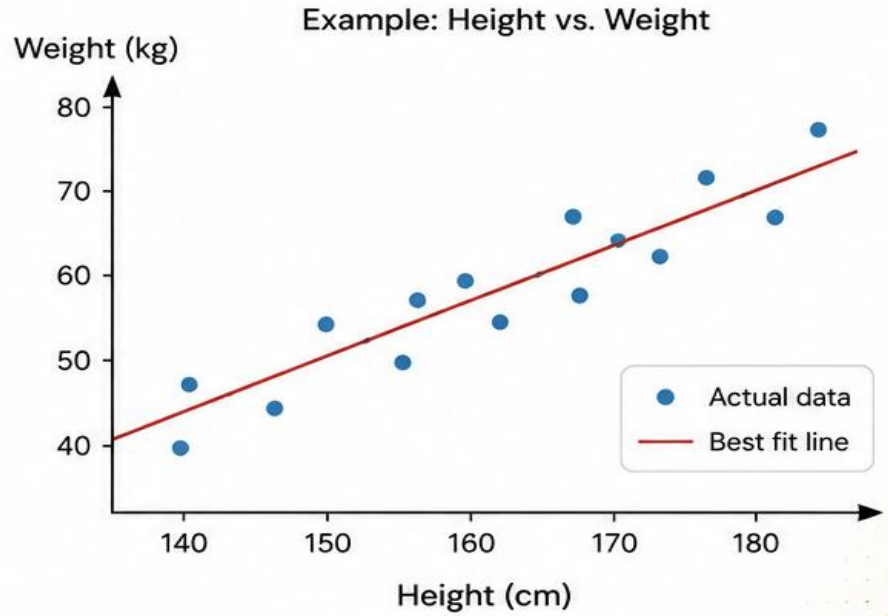
- Traditional interpretable algorithms are inherently transparent by construction (intrinsic interpretability).
- Covered in this section: linear regression, logistic regression, GLMs, GAMs, naive Bayes, Bayesian networks, decision trees, and rule induction.
- Tools/Libraries: scikit-learn, statsmodels, pyGAM, pgmpy (Bayesian networks), Orange/wittgenstein (rule induction).

Linear Regression

Linear regression is a simple method used to predict a number (called the target) using one or more other numbers (called features).
It works by finding the **best straight line** that has the least error from the actual data.

In simple words:
Linear regression tries to find a straight line that best fits the data points.

- Suppose we have d features (inputs): $x_1, x_2, \ldots, x_d$
- To make calculations easy, we add one more input called bias term x_0, which is always 1.
- So, the input becomes: $x = [1, x_1, x_2, \ldots, x_d]$
- We learn (find) weights: $w = [w_0, w_1, w_2, \ldots, w_d]$
- The predicted (or estimated) value is a combination of these inputs and weights.



The Linear Model (Formula)

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_d x_d$$

$\hat{y}$	= predicted value (our answer)
$x_1, x_2, \ldots, x_d$	= input features
w_0	= weight for bias term (intercept)
$w_1, w_2, \ldots, w_d$	= weights for features

Interpreting Linear Regression Model

After training a linear regression model, we can understand how the model works and what the results mean.

1. Equation of the Line

The model finds a straight line that best fits the data.

$$y = w_0 + w_1 x$$

- y = predicted value (output)
- x = input feature
- w_0 = intercept (value of y when $x = 0$)
- w_1 = slope or weight (change in y for 1 unit change in x)

2. Meaning of Slope (w_1)

It tells us how much the output (y) changes when the input (x) increases by 1 unit.

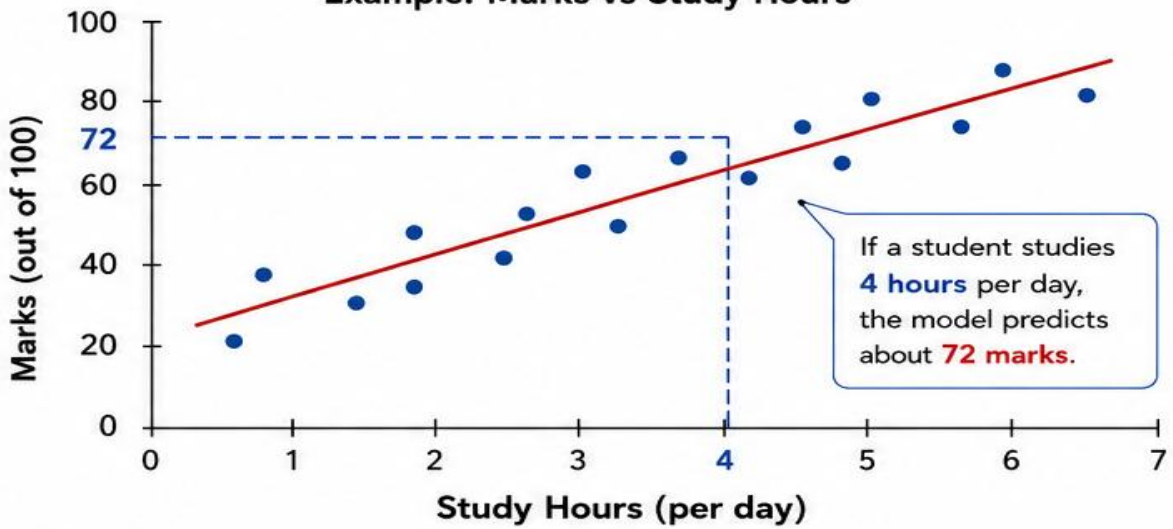
If $w_1 = 10$, it means for every 1 extra hour of study, marks increase by 10 (on average).

3. Meaning of Intercept (w_0)

It tells us the predicted value of y when $x = 0$.

If $w_0 = 20$, it means even if a student does not study at all ($x = 0$), the model predicts 20 marks (maybe from guessing or basic knowledge).

Example: Marks vs Study Hours



4. Goodness of Fit (R^2)

R^2 tells us how well the line fits the data.

R^2 close to 1 → Model is very good (explains most of the data)

R^2 close to 0 → Model is not good (explains little of the data)

5. Use of the Model

- We can use the line to make predictions for new values.
- Example: For 6 hours of study per day, the model can predict marks.
- Helps in decision making and understanding the relationship between input and output.

Explainable Properties of Linear Regression



Linear regression is one of the most interpretable machine learning models.
It has simple and explainable properties that help us understand how it works.

Properties	Values	
Local or global	Global	The model explains the entire dataset, not just a small part.
Linear or non-linear	Linear	It fits a straight line (or a hyperplane in higher dimensions).
Monotonic or non-monotonic	Monotonic	If a feature increases, the output either increases or decreases, not both.
Feature interactions captured	No	Linear regression does not automatically capture interactions between features.
Model complexity	Low	The model is simple, easy to train, and easy to understand.

Logistic Regression



- Linear regression is used for predicting **continuous values** (like marks, price, temperature).
- But in many real-life problems, we only want to predict a **category** (like Yes/No, True/False, Pass/Fail).
- Logistic regression is a method used for **classification problems**.
- It gives the **probability** that a data point belongs to a particular class (between 0 and 1).



Logistic regression first calculates a linear combination of inputs (like study hours, test score, etc.)
Then it passes this value through a special function called the **sigmoid function**
to convert it into a probability between 0 and 1.

1. Linear Combination

$$z = w^T x + b$$

→

2. Sigmoid (Logistic) Function

$$h(x) = \frac{1}{1 + e^{-z}}$$

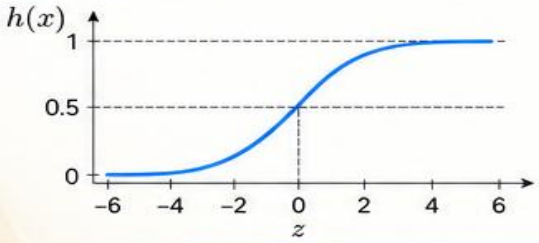
Where,

- **x** = input feature vector
- **w** = weight vector (learned by the model)
- **b** = bias or intercept
- **h(x)** = probability (between 0 and 1)

How it works

Input Features
Study hours
Test score
Attendance
...

→ **Linear Combination** $z = w^T x + b$ → **Sigmoid Function** $h(x) = \frac{1}{1 + e^{-z}}$ → **Probability** (0 to 1) e.g., 0.85



- As z increases, $h(x)$ approaches 1.
- As z decreases, $h(x)$ approaches 0.
- When $z = 0$, $h(x) = 0.5$

Example: Medical Test



Patient Information
(Age, Symptoms, Test Results, etc.)

→



Logistic Regression Model

→

0.85
Probability of Disease = Yes

The model gives a probability of 0.85, which means there is an **85% chance** that the patient has the disease.

Interpretability of Logistic Regression

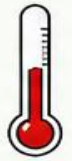
Logistic regression helps us understand how each input (feature) affects the odds of an outcome happening.

1 Continuous Features



When a continuous feature (like age, height, temperature) increases by one unit, the odds of the outcome change by a factor of $exp(w_i x_i)$.

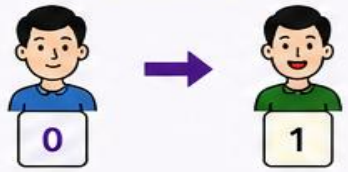
Example:



If the coefficient for temperature is 0.3, then for each 1°C increase, the odds multiply by $exp(0.3) \approx 1.35$.

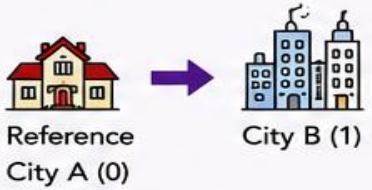
So, 1°C higher means 35% higher odds.

2 Categorical Features



Categorical features (like Gender, City, etc.) are turned into 0 and 1 (one-hot encoding). When a category changes from the reference (0) to the other category (1), the odds change by a factor of $exp(w_i x_i)$.

Example:



If the coefficient for City B is 0.7, then choosing City B instead of City A multiplies the odds by $exp(0.7) \approx 2.01$.

So, odds are about 2 times higher in City B.

3 Intercept (Constant)



The intercept (b) is the log-odds when all continuous features are 0 and all categories are at their reference (0). It represents the **baseline odds**.

Example:



Age = 0
Income = 0
City = Reference
Gender = Reference

The model output at this point (intercept) gives the estimated odds.

This is the starting point before any effects.

Explainable Properties of Logistic Regression

Properties	Values
Local or global The interpretation applies to the entire model, not to specific instances.	**Global** The explanation is valid for all predictions made by the model.
Linear or non-linear Relationship between the input features and the log-odds.	**Linear** Logistic regression is a linear model in the log-odds space.
Monotonic or non-monotonic Direction of relationship between a feature and the outcome.	**Monotonic** A feature's effect on the log-odds is always in one direction (increasing or decreasing), based on the sign of its coefficient.
Feature interactions captured Whether the model automatically learns interactions between features.	**No** Logistic regression does not capture interactions unless they are explicitly created as new features.
Model complexity Overall complexity of the model and its interpretability.	**Medium** More complex than linear regression but still easy to interpret.

Generalized Linear Models (GLM)

- Extend linear regression to non-normal response distributions (Poisson, Binomial, Gamma) via a link function $g(\cdot)$: $g(E[y]) = \beta 0 + \beta 1x1 + \ldots$
- Common link functions: identity (linear regression), logit (logistic regression), log (Poisson regression).
- Retains coefficient-based interpretability while flexibly modeling diverse data types (counts, rates, proportions).

GENERALIZED LINEAR MODEL (GLM) – EXPLAINED WITH EXAMPLE

GLM generalizes linear regression by using a link function to relate predictors to the expected value of the outcome.

1. What is GLM?

GLM has three components:

1. **Random component (Y)**
   The outcome variable follows a distribution from the exponential family.
2. **Systematic component (η)**
   A linear combination of predictors.
3. **Link function (g)**
   Connects the expected value of Y to the linear predictor.

GLM Formula

$$g(\mu) = \eta = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

where,

μ = $E(Y \mid X_1, X_2, \ldots, X_p)$

$g()$ = link function

β's = model coefficients

2. Example: Customer Purchase (Logistic Regression)

Problem: Predict whether a customer will buy a product (Yes = 1, No = 0) based on Age and Annual Income.

Outcome (Y): Purchase (1 = Yes, 0 = No)

Predictors (X):

- X_1 = Age (in years)
- X_2 = Annual Income (in $1000)

Model (Logistic Regression)

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 \text{Age} + \beta_2 \text{Income}$$

where $p = P(Y = 1 \mid \text{Age, Income})$



3. Other GLM Examples

Linear Regression (Normal)

Outcome: House Price (continuous)

Model: $\mu = \beta_0 + \beta_1 \text{Area} + \beta_2 \text{Bedrooms} + \beta_3 \text{Age}$

Link: $g(\mu) = \mu$ (Identity)



Poisson Regression (Count)

Outcome: Number of accidents (count)

Model: $\log(\mu) = \beta_0 + \beta_1 \text{Traffic Volume}$

Link: $g(\mu) = \log(\mu)$ (Log link)



Gamma Regression (Positive Continuous)

Outcome: Insurance claim amount (positive)

Model: $\log(\mu) = \beta_0 + \beta_1 \text{Age} + \beta_2 \text{Claim Type}$

Link: $g(\mu) = \log(\mu)$ (Log link)



4. Effect of Feature Value Variation (Keeping Other Features Fixed)

A. Linear Regression (Identity Link)

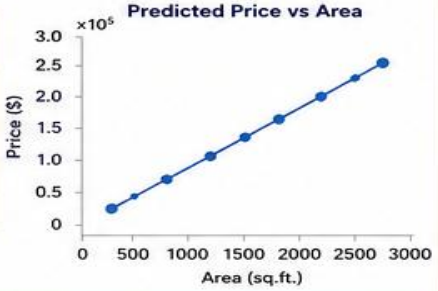
Model: $\mu = \beta_0 + \beta_1 X$

(Example: House Price vs Area)



Effect: When Area changes, the predicted house price changes linearly.

Area (sq.ft.)	Predicted Price ($)
500	80,000
1000	120,000
1500	160,000
2000	200,000
2500	240,000



Interpretation:
For every additional 500 sq.ft., the price increases by a constant amount (= $\beta_1 \times 500$). The effect of Area on Price is linear and constant.

B. Logistic Regression (Logit Link)

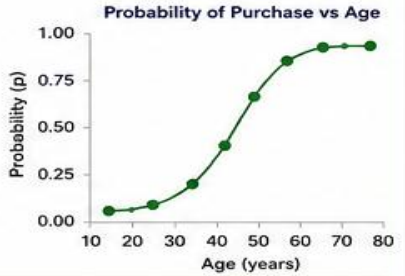
Model: $\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 \text{Age}$

(Example: Purchase Probability vs Age)



Effect: When Age increases, the probability of purchase changes in an S-shaped (sigmoid) curve.

Age (years)	Prob. of Purchase (p)
20	0.09
30	0.24
40	0.50
50	0.76
60	0.91
70	0.97



Interpretation:
As Age increases, the log-odds increase linearly, but the probability increases slowly at first, then faster around the middle, and then levels off near 1.

C. Poisson Regression (Log Link)

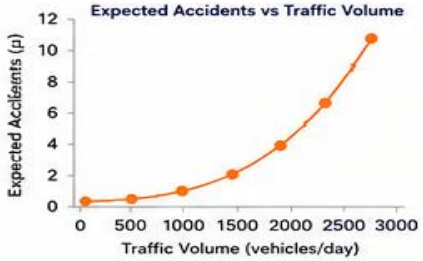
Model: $\log(\mu) = \beta_0 + \beta_1 \text{Traffic Volume}$

(Example: Expected Accidents vs Traffic Volume)



Effect: When Traffic Volume increases, the expected number of accidents increases exponentially.

Traffic Volume (vehicles/day)	Expected Number of Accidents (μ)
500	0.82
1000	1.35
1500	2.23
2000	3.69
2500	6.10
3000	10.07



Interpretation:
As Traffic Volume increases, the expected count of accidents grows exponentially (multiplicative effect).

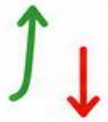
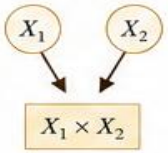
Key Takeaway

GLM uses an appropriate link function to model different types of outcomes. Feature value changes affect the expected outcome linearly (Linear), sigmoidally (Logistic), or exponentially (Poisson).

Interpretable · Flexible · Predictive · Reliable

Explainable Properties of Generalized Linear Models (GLM)

GLMs include Linear Regression (Normal), Logistic Regression (Bernoulli) and Poisson Regression.

Property	Values (GLM)	Explanation	Example
Local or global The interpretation applies to the entire model, not to specific instances.	**Global** Same coefficients explain every prediction. 	A single set of coefficients (β) is used for all observations.	**Example (Logistic Regression)** $\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$ Applies to every case.
Linear or non-linear Relationship between the linear predictor and features is linear (in the link scale).	**Linear (in the link scale)** Linear combination of features ($\eta = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$). 	The linear predictor (η) is linear in features; the link function can make the final relationship non-linear.	**Link functions** Linear Regression (Normal) : $g(\mu) = \mu$ Logistic Regression (Bernoulli) : $g(p) = \log\left(\frac{p}{1-p}\right)$ Poisson Regression : $g(\mu) = \log(\mu)$
Monotonicity Relationship between a feature and the output is monotonic based on the sign of its coefficient.	**Monotonic** Positive $\beta \rightarrow$ output/probability increases; Negative $\beta \rightarrow$ output/probability decreases (holding others constant). 	In GLMs, a feature has a consistent direction of effect on the expected output/ probability.	**Example (Logistic Regression)** $\beta_1 = 0.80 \rightarrow$ Odds increase $\beta_2 = -0.60 \rightarrow$ Odds decrease
Feature interactions captured Standard GLM does not capture interactions unless they are explicitly included.	**No (by default)** Can capture interactions only when we add interaction terms (e.g., $X_1 \times X_2$). 	Model includes only main effects unless interaction terms are added manually.	**Example** $\eta = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 (X_1 X_2)$ Here, β_3 captures the interaction between X_1 and X_2.
Model complexity GLMs are relatively simple and efficient.	**Low / Medium** Few parameters; easier to train and interpret. 	Complexity depends on the number of features and added terms.	Compared to tree ensembles, GLMs are simpler and easier to explain.

Generalized Additive Models (GAM)

- Extend GLMs by replacing linear terms with smooth, flexible functions: $g(E[y]) = \beta 0 + f1(x1) + f2(x2) + \ldots$
- Each fi is typically a spline, allowing non-linear relationships while keeping each feature's effect separately visualizable.
- Captures non-linearity (unlike plain linear/logistic regression) while preserving additive, per-feature interpretability.

Generalized Additive Model (GAM) – Explained with Example

GAM extends GLM by allowing Non-linear relationships between predictors and the outcome using smooth functions.

1. What is GAM?

GAM has three components:

1. **Random component (Y)**
   The outcome variable follows a distribution from the exponential family.
2. **Systematic component**
   Additive combination of smooth functions of predictors.
3. **Link function (g)**
   Connects the expected value of Y to the additive predictor.

GAM Formula

$$g(\mu) = \eta = \beta_0 + \sum_{j=1}^{p} f_j(X_j)$$

where,

$\mu = E(Y \mid X_1, X_2, \ldots, X_p)$

$g()$ = link function

$f_j()$ = smooth function for X_j

β_0 = intercept

2. Example: House Price (GAM with Gaussian Family)

Goal: Predict House Price based on Area, Bedrooms and Age of house.

Response (Y): Price (in $1000)

Predictors:

- X_1 = Area (sq.ft.)
- X_2 = Number of Bedrooms
- X_3 = Age of House (years)



Model (Identity Link for Gaussian):

$$Price = \beta_0 + f_1(\text{Area}) + f_2(\text{Bedrooms}) + f_3(\text{Age}) + \varepsilon$$

Where f_1, f_2, f_3 are smooth (non-linear) functions estimated from data.

3. Other GAM Examples

Binomial GAM (Logistic Link)

Outcome: Purchase (Yes/No)

Model: $\log\left(\frac{p}{1-p}\right) = \beta_0 + f_1(\text{Age}) + f_2(\text{Income})$

Poisson GAM (Log Link)

Outcome: Number of claims (count)

Model: $\log(\mu) = \beta_0 + f_1(\text{Exposure}) + f_2(\text{Driver Age})$

Gamma GAM (Log Link)

Outcome: Insurance claim amount (positive)

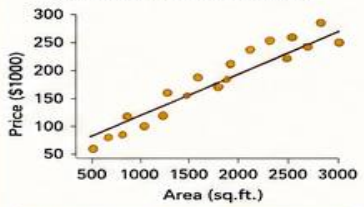
Model: $\log(\mu) = \beta_0 + f_1(\text{Age}) + f_2(\text{Claim Type})$

4. Effect of Feature Value Variation (Keeping Other Features Fixed)

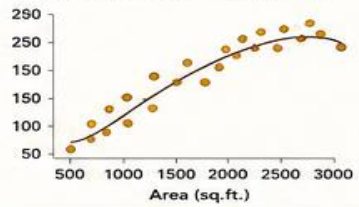
A. Linear (GLM) vs GAM for Area (House Price)

Response: Price **Link:** Identity Other features fixed at their mean

1) GLM (Linear Relationship)



2) GAM (Non-linear Relationship)



Effect: Price changes at a constant rate as Area increases.

Effect: Price increases with Area up to a point, then the effect slows down (non-linear).

Area vs Predicted Price (GLM)	
Area (sq.ft.)	Predicted Price ($1000)
500	75
1000	100
1500	125
2000	150
2500	175
3000	200

Area vs Predicted Price (GAM)	
Area (sq.ft.)	Predicted Price ($1000)
500	60
1000	100
1500	150
2000	185
2500	200
3000	205

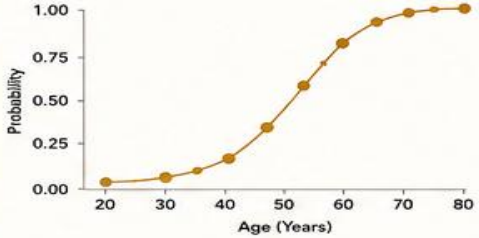
Interpretation: In GLM, each additional 500 sq.ft. adds the same amount to price. In GAM, the increase is larger in mid-range and smaller at high area.

B. Logistic GAM: Effect of Age on Purchase Probability

Response: Purchase (Yes=1, No=0) **Link:** Logit Income fixed at mean

Age vs Predicted Probability (GAM)	
Age (Years)	P(Purchase = 1)
20	0.05
30	0.12
40	0.30
50	0.60
60	0.82
70	0.92
80	0.95

Probability of Purchase vs Age



Effect: As Age increases, the probability of purchase increases slowly at first, then rapidly around middle age, and then levels off.

Comparison with GLM (Linear in log-odds)

Age	20	30	40	50	60	70	80
GLM Prob.	0.09	0.17	0.29	0.45	0.64	0.78	0.87
GAM Prob.	0.05	0.12	0.30	0.60	0.82	0.92	0.95

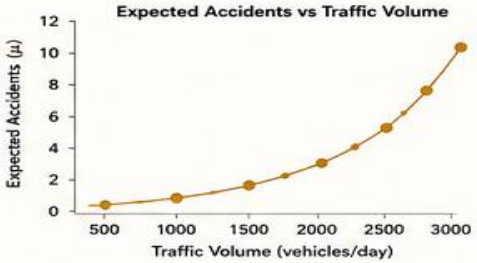
GLM changes probability at a constant rate in log-odds, while GAM captures the non-linear (S-shaped) effect.

C. Poisson GAM: Effect of Traffic Volume on Accidents

Response: Number of Accidents (count) **Link:** Log Driver Age fixed at mean

Traffic Volume vs Expected Accidents (GAM)	
Traffic Volume (vehicles/day)	Expected Accidents (μ)
500	0.40
1000	0.85
1500	1.60
2000	2.80
2500	4.80
3000	7.80

Expected Accidents vs Traffic Volume



Effect: As Traffic Volume increases, the expected number of accidents grows slowly at first, then faster (exponential-like).

Comparison with GLM (Linear in log-scale)

Traffic Volume	500	1000	1500	2000	2500	3000
GLM (μ)	0.35	0.70	1.40	2.80	5.60	11.20
GAM (μ)	0.40	0.85	1.60	2.80	4.80	7.80

GLM overestimates at high volume because it assumes constant exponential growth rate, while GAM adapts the rate.

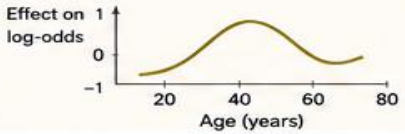
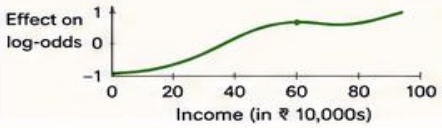
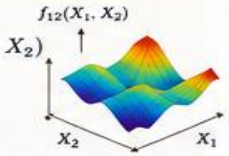
Key Takeaway

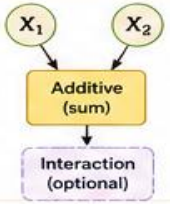
GAM allows each predictor to have its own flexible (non-linear) effect on the outcome.
Feature value variation changes the expected outcome according to the learned smooth function.

Interpretable Flexible Predictive Reliable

Explainable Properties of Generalized Additive Models (GAM)

GAMs extend GLMs by allowing non-linear (smooth) effects of features while keeping the model highly interpretable.

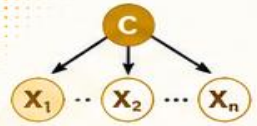
Property	Values (GAM)	Explanation	Example
Local or global Both global and local interpretability.	**Global + Local** Overall effect of each feature is global; local explanations are available via partial effects.	Each smooth function shows the global relationship of a feature with the outcome. For a specific prediction, we can see how each feature contributed (local).	**Example** $Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p)$ Sum of smooth functions.
Linear or non-linear Captures non-linear relationships through smooth functions.	**Non-linear (additive)** Each feature has its own smooth function $f_j(X_j)$. The model is additive (no product terms by default).	Relationships do not need to be linear. Each feature can have a flexible (non-linear) effect on the outcome.	**Example: $f_{Age}(x)$**  The effect of Age on the outcome is non-linear.
Monotonicity Smooth functions can be constrained to be monotonic if required.	**Up to the modeler** Functions can be unconstrained (flexible) or constrained to be monotonic increasing/decreasing.	We can enforce monotonicity when we have domain knowledge that the effect should only increase or decrease.	**Example (Monotonic effect of Income)**  Effect is constrained to increase with Income.
Feature interactions captured Not by default; interactions can be added explicitly.	**Additive by default** GAMs include only main effects: $f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p)$. Interactions require adding term $f_{ij}(X_i, X_j)$.	Interpretability remains high because the default model is additive. Any interaction included is explicit and interpretable.	**Example: Adding interaction** $Y = \beta_0 + f_1(X_1) + f_2(X_2) + f_{12}(X_1, X_2)$ The surface plot shows the interaction between X_1 and X_2. 
Model complexity Low to medium; controlled by the number of smooth terms and their flexibility.	**Low / Medium** Controlled by the basis size (k) for each smooth function.	More flexible functions (larger k) increase complexity, but GAMs are still simpler and easier to interpret than black-box models.	**Example** $k = 5$ (low flexibility) → very smooth $k = 20$ (high flexibility) → captures more wiggles



Key Takeaway

GAMs are highly interpretable because each feature's effect is shown as a smooth function. They capture non-linear patterns while remaining transparent, additive, and easier to explain than complex black-box models.

Transparent · Flexible · Interpretable



Naive Bayes (Binary Features) – Explained with Example

Naive Bayes is a probabilistic classifier based on Bayes' Theorem with an assumption of conditional independence between features.

Bayes' Theorem

$$P(C \mid X) = \frac{P(C)P(X \mid C)}{P(X)}$$

1. What is Naive Bayes?

Naive Bayes has three components:

1. **Class (C)**
   Target variable (e.g., Spam / Not Spam).
2. **Features ($X_1, X_2, \ldots, X_n$)**
   Input variables. Here each feature is binary (1 = present/yes, 0 = absent/no).
3. **Conditional Independence Assumption**
   All features are assumed independent of each other given the class.

Naive Bayes Formula

$$P(C \mid X_1, X_2, \ldots, X_n) \propto P(C) \prod_{i=1}^{n} P(X_i \mid C)$$

where,

- $P(C)$ = prior probability of class C
- $P(X_i \mid C)$ = likelihood of feature X_i given class C
- $P(X)$ = evidence (same for all classes)

Naive Bayes Formula (for any type of features)

$$P(C \mid X_1, X_2, \ldots, X_n) \propto P(C) \prod_{i=1}^{n} P(X_i \mid C)$$

Predict the class with the highest posterior probability.

2. Example: Email Spam Classification (Binary Features Only)

Goal: Classify an email as Spam (S) or Not Spam (H) based on the presence of certain words.

Features (All Binary)

- X_1 = Has word "Free"
- X_2 = Has word "Win"
- X_3 = Has word "Money"
- X_4 = Has word "Project"

→ **Predict** Spam (S) or Not Spam (H)

Training Data (Example)

Email	Class (C)	Free (X_1)	Win (X_2)	Money (X_3)	Project (X_4)
E_1	Spam	1	1	1	0
E_2	Spam	1	1	0	0
E_3	Spam	0	1	1	0
E_4	Spam	0	0	1	1
E_5	Spam	0	1	0	1
E_6	Spam	0	0	0	0
E_7	Not Spam	0	0	1	1
E_8	Not Spam	0	1	1	0
E_{10}	Not Spam	0	1	0	0
E_{11}	Not Spam	1	0	0	1
E_{12}	Not Spam	0	0	0	0

Priors (from training data)

$P(S) = 6/12 = 0.50$

$P(H) = 6/12 = 0.50$

Conditional Probabilities (Computed from data)

Given Spam (S) (6 emails)

Feature	$P(X_i = 1 \mid S)$	$P(X_i = 0 \mid S)$
Free (X_1)	4/6 = 0.667	2/6 = 0.333
Win (X_2)	4/6 = 0.667	2/6 = 0.333
Money (X_3)	4/6 = 0.667	2/6 = 0.333
Project (X_4)	2/6 = 0.333	4/6 = 0.667

Given Not Spam (H) (6 emails)

Feature	$P(X_i = 1 \mid S)$	$P(X_i = 0 \mid S)$
Free (X_1)	1/6 = 0.167	5/6 = 0.833
Win (X_2)	1/6 = 0.167	5/6 = 0.833
Money (X_3)	2/6 = 0.333	4/6 = 0.667
Project (X_4)	3/6 = 0.500	3/6 = 0.500

3. Predict a New Email (Binary Features)

New Email to classify:

Feature	Free (X_1)	Win (X_2)	Money (X_3)	Project (X_4)
Value	1	1	0	1

Step 1: Compute unnormalized posterior for Spam (S)

$$P(S) \times P(X_1 = 1 \mid S) \times P(X_2 = 1 \mid S) \times P(X_3 = 0 \mid S) \times P(X_4 = 1 \mid S)$$

$$= 0.50 \times 0.667 \times 0.667 \times 0.333 \times 0.333$$

$$= 0.0247$$

Step 2: Compute unnormalized posterior for Not Spam (H)

$$P(H) \times P(X_1 = 1 \mid H) \times P(X_2 = 1 \mid H) \times P(X_3 = 0 \mid H) \times P(X_4 = 1 \mid H)$$

$$= 0.50 \times 0.167 \times 0.167 \times 0.667 \times 0.500$$

$$= 0.0046$$

Step 3: Compare

$P(S \mid X) \propto 0.0247$ $\qquad$ $P(H \mid X) \propto 0.0046$

Since 0.0247 > 0.0046 ⇒ Predict: Spam (S)

4. Effect of Changing Each Binary Feature (Keeping Others Fixed)

A. Change "Free" (X_1)

(Other features fixed: Win=1, Money=0, Project=1)

Free (X_1)	$P(S \mid X)$	$P(H \mid X)$	Predicted Class
0 (Absent)	0.0062	0.0389	Not Spam (H)
1 (Present)	0.0247	0.0046	Spam (S)

Interpretation: Having word "Free" increases the likelihood of Spam and changes the prediction.

B. Change "Win" (X_2)

(Other features fixed: Free=1, Money=0, Project=1)

Win (X_2)	$P(S \mid X)$	$P(H \mid X)$	Predicted Class
0 (Absent)	0.0062	0.0389	Not Spam (H)
1 (Present)	0.0247	0.0046	Spam (S)

Interpretation: Having word "Win" increases the likelihood of Spam and changes the prediction.

C. Change "Money" (X_3)

(Other features fixed: Free=1, Win=1, Project=1)

Money (X_3)	$P(S \mid X)$	$P(H \mid X)$	Predicted Class
0 (Absent)	0.0247	0.0046	Spam (S)
1 (Present)	0.0988	0.0015	Spam (S)

Interpretation: Having word "Money" further increases the likelihood of Spam.

D. Change "Project" (X_4)

(Other features fixed: Free=1, Win=1, Money=0)

Project (X_4)	$P(S \mid X)$	$P(H \mid X)$	Predicted Class
0 (Absent)	0.0494	0.0093	Spam (S)
1 (Present)	0.0247	0.0046	Spam (S)

Interpretation: Presence of word "Project" slightly increases Spam likelihood but prediction remains Spam.

Key Takeaway

- Naive Bayes multiplies prior with conditional probabilities of all features.
- For binary features, we use $P(X = 1 \mid C)$ and $P(X = 0 \mid C)$.
- Changing any binary feature (keeping others fixed) changes the product, which changes the posterior probability and may change the predicted class.

Interpretable · Simple · Probabilistic · Effective with Small Data

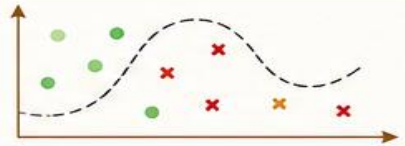
Explainable Properties of Naïve Bayes Classifier

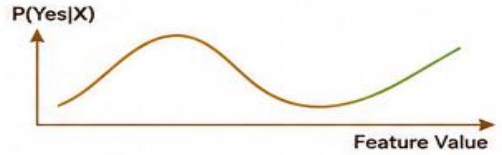
Properties	Values
Local or global Probability tables provide a global model view; individual feature probabilities explain a particular prediction.	**Global + Local** The model's probability tables are global. For a specific case, we can explain the prediction using the feature probabilities (local explanation).
Linear or non-linear Probability multiplication can produce non-linear decision boundaries.	**Non-linear** The combination of probabilities can lead to complex (non-linear) decision boundaries.
Monotonicity Increasing a feature does not automatically guarantee that the prediction probability always increases or decreases.	**Not necessarily monotonic** The relationship between a feature and the predicted probability may increase or decrease depending on the data distribution.
Feature interactions Naïve Bayes assumes conditional independence of features given the class.	**No explicit interactions** The model does not learn interactions between features unless we create new features explicitly.
Model complexity Mainly consists of class priors and conditional probability estimates.	**Low** Simple structure with few parameters, easy to train and easy to understand.
Prediction mechanism Calculates the probability of each class given the observed features.	**Probabilistic** Uses Bayes' theorem to compute P(Class | Features) for all classes.
Feature contribution Each feature contributes through its class-conditional probability.	**Yes** We can see how each feature supports or opposes a class using its probability.

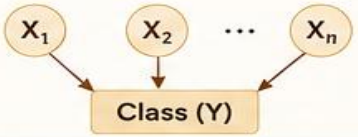
P(Feature | Class)

Feature	Class = Yes	Class = No
Glucose	0.80	0.20
BMI	0.60	0.40
Age	0.55	0.45
...	...	...

P(Yes|X)=0.82
P(No|X)=0.18



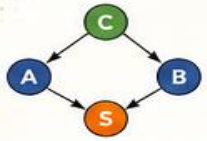






$$P(C_k|X) = \frac{P(X|C_k)P(C_k)}{\sum_j P(X|C_j)P(C_j)}$$





Bayesian Networks – Explained with Example

Bayesian Networks (BNs) are graphical models that represent probabilistic relationships among variables.
They combine conditional dependencies (structure) with probabilities (CPTs).

Root / Parent
Intermediate
Child / Target

1. What is a Bayesian Network?

A BN is a Directed Acyclic Graph (DAG) where:

- Nodes = Random Variables
- Arrows = Conditional Dependencies

Joint Probability Factorization

The joint probability of all variables is the product of each node's probability given its parents.

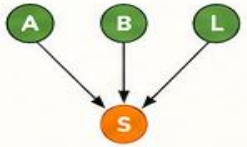
$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$

Benefits

- Captures conditional dependencies
- Handles missing data
- Interpretable and modular

2. Example: Email Spam Detection

Goal: Predict whether an email is Spam (S) based on other factors.



Variables:

- A Contains "Free"? (A = 1: Yes, 0: No)
- B Contains "Win"? (B = 1: Yes, 0: No)
- L Large Attachment? (L = 1: Yes, 0: No)
- S Spam? (S = 1: Spam, 0: Not Spam)

Structure (DAG)

A, B, L are parent nodes.
S (Spam) is the child.
Assumption: A, B, L are conditionally independent given S.

Conditional Probability Tables (CPTs)

Priors for Parents	
Variable	P(= 1)
P(A = 1)	0.30
P(B = 1)	0.20
P(L = 1)	0.10

Conditional Probability of S given A, B, L				
A	B	L	P(S = 1 | A, B, L)	P(S = 0 | A, B, L)
0	0	0	0.01	0.99
1	0	0	0.05	0.95
0	1	0	0.06	0.94
0	0	1	0.08	0.92
1	0	0	0.30	0.70
1	0	1	0.35	0.65
0	1	0	0.40	0.60
1	1	1	0.90	0.10

Example Inference

Find $P(S = 1 \mid A = 1, B = 1, L = 0)$

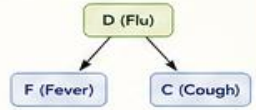
From CPT:
$P(S = 1 \mid 1, 1, 0) = 0.30$
$P(S = 0 \mid 1, 1, 0) = 0.70$

⇒ Predict: Not Spam (Higher prob. = 0.70)

3. Other Bayesian Network Examples

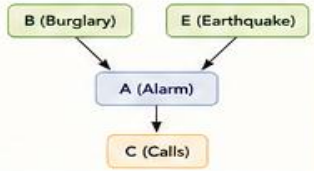
Medical Diagnosis

Variables: Fever (F), Cough (C), Flu (D)
(F and C depend on D)



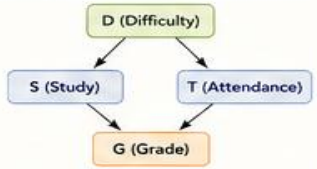
Burglary Alarm

Variables: Burglary (B), Earthquake (E), Alarm (A), Calls (C)
A depends on B and E;
C depends on A.



Student Performance

Variables: Study (S), Attendance (T), Difficulty (D), Grade (G)
S and T depend on D;
G depends on S and T.

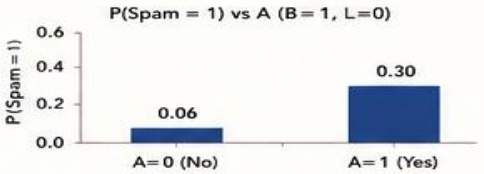


4. Effect of Feature Value Variation on P(Spam = 1) (Keeping Other Features Fixed)

A. Vary "Free" (A)

Keep B = 1, L = 0 fixed

A (Free)	B	L	P(S = 1 | A, B, L)	P(S = 0 | A, B, L)
0 (No)	1	0	0.06	0.94
1 (Yes)	1	0	0.30	0.70

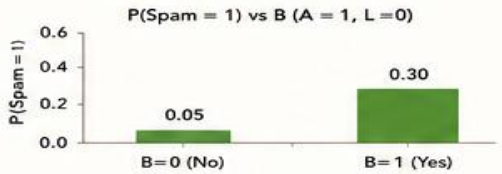


Interpretation: If the email contains "Free", the probability of Spam increases from 0.06 to 0.30.

B. Vary "Win" (B)

Keep A = 1, L = 0 fixed

B (Win)	A	L	P(S = 1 | A, B, L)	P(S = 0 | A, B, L)
0 (No)	1	0	0.05	0.95
1 (Yes)	1	0	0.30	0.70

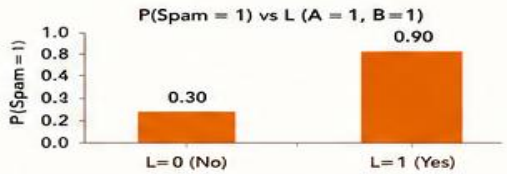


Interpretation: If the email contains "Win", the probability of Spam increases from 0.05 to 0.30.

C. Vary "Large Attachment" (L)

Keep A = 1, B = 1 fixed

L (Attach.)	A	B	P(S = 1 | A, B, L)	P(S = 0 | A, B, L)
0 (No)	1	1	0.30	0.70
1 (Yes)	1	1	0.90	0.10



Interpretation: If the email has a large attachment, the probability of Spam jumps from 0.30 to 0.90.

D. Joint Effect of Features

Compare combinations (Joint effect)

A	B	L	P(S = 1 | A, B, L)
0	0	0	0.01
1	0	0	0.05
0	1	0	0.06
0	1	1	0.06
1	1	0	0.30
1	1	1	0.90

Low Spam Probability
High Spam Probability

Interpretation: More suspicious features together greatly increase the probability of Spam.

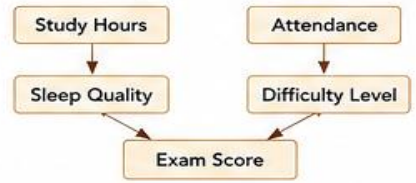
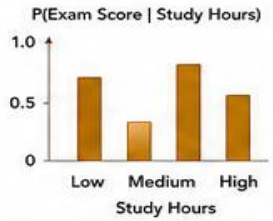
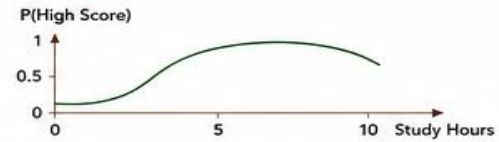
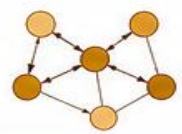
Key Takeaway

- Bayesian Networks represent dependencies and compute probabilities using CPTs.
- Each feature value change (keeping others fixed) changes the conditional probability of the target.
- They are interpretable and help in reasoning under uncertainty.

Interpretable · Probabilistic · Handles Missing Data · Modular & Scalable

Explainable Properties of Bayesian Networks

Bayesian Networks (BNs) are graphical probabilistic models that represent variables and their conditional dependencies.

Property	Values (Bayesian Networks)	Explanation	Example
Local or global Both global structure and local conditional probabilities are interpretable.	**Global + Local** The entire network structure is global; CPDs (Conditional Probability Distributions) explain local relationships. 	We can understand the overall dependencies (global) and also how each variable behaves given its parents (local).	**Example: Student Performance Network**  We can see all variables and dependencies at a glance.
Linear or non-linear Captures both linear and non-linear relationships via conditional probability tables.	**Non-linear** Relationships are represented using probability distributions (CPDs), not restricted to linear forms. 	A variable's effect on another can be any shape—captured in its CPD.	**Example: CPD: Exam Score | Study Hours, Difficulty Level** See table below.
Monotonicity Monotonic relationships are not assumed but can be enforced or learned from data.	**Not necessarily monotonic** Effects can increase, decrease or be non-monotonic depending on the CPD. 	The probability of a high score may increase and then plateau or change depending on other variables.	**Example: P(High Score | Study Hours, Difficulty=Medium)** 
Feature interactions captured Interactions are naturally represented through the network structure.	**Yes** Dependencies and interactions are explicit in the graph. 	If two variables both influence the same child, an interaction is captured (even if not numerical).	**Example: Interaction**  The effect of Study Hours depends on Difficulty Level and vice-versa.
Model complexity Generally low to medium for moderate networks.	**Low / Medium** Complexity grows with number of variables, parents per node, and CPD sizes. 	BNs are simpler and more interpretable than many black-box models, especially when the network is sparse.	**Example** A network with 6 variables and small CPDs is easy to understand and maintain. 

Study Hours	Difficulty Level	P(Exam Score)		
		Low (0–40)	Medium (40–70)	High (70–100)
Low	High	0.70	0.25	0.05
Low	Medium	0.40	0.40	0.20
High	High	0.10	0.30	0.60
High	Medium	0.05	0.25	0.70

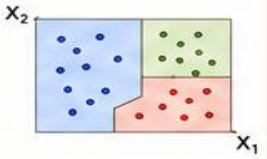
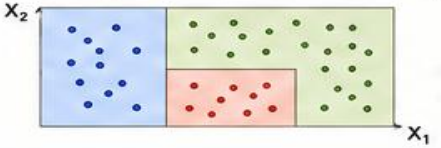
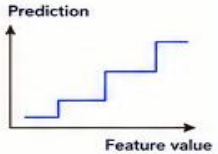
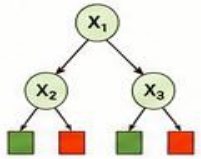
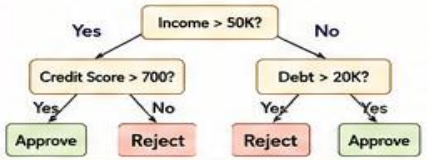
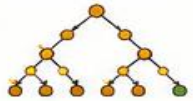
Key Takeaway

Bayesian Networks are highly interpretable because they provide a clear visual structure of dependencies, capture interactions naturally, and support uncertainty quantification.

Transparent · Structured · Quantifiable · Reliable

Explainable Properties of Decision Trees

Decision Trees are simple, visual models that split data based on feature conditions, making them highly interpretable.

Property	Values (Decision Trees)	Explanation	Example / Visual
Local or global The entire tree is a global model; each path from root to leaf explains a local prediction.	**Global + Local** The whole tree structure is global. Each root-to-leaf path provides a local explanation for a specific prediction. 	We can understand the overall decision logic (global) and also explain an individual prediction (local) by tracing the path followed.	**Example: Loan Approval Tree**  A prediction is made by following a single path from root to a leaf.
Linear or non-linear Can capture both linear and non-linear relationships through recursive partitioning.	**Non-linear** Decision boundaries are axis-aligned but overall boundaries can be highly non-linear. 	By splitting the feature space into rectangles, trees can model complex patterns and non-linear relationships.	**Example: Non-linear Decision Boundary**  The final decision boundary (union of rectangles) is non-linear.
Monotonicity Not inherently monotonic, but monotonicity can be enforced using constraints or pruning.	**Not necessarily monotonic** The prediction can increase, decrease or stay the same as a feature increases. 	As a feature increases, prediction may change up or down multiple times depending on the splits.	**Example** Feature (x): 0 1 2 3 4 5 6 7 8 Prediction (Y): 0 0 1 1 0 1 1 0 0 Prediction changes multiple times when feature increases.
Feature interactions captured Interactions are captured automatically through hierarchical splits.	**Yes (Implicitly)** Splits on one feature can depend on previous splits on other features (hierarchical structure). 	If a split on X_2 occurs after a split on X_1, it means the effect of X_2 depends on the value of X_1 (interaction).	**Example: Interaction**  Credit Score matters only when Income > 50K, showing an interaction.
Model complexity Complexity depends on tree depth, number of leaves and features.	**Low / Medium** A small tree is simple and highly interpretable; deep trees are complex and harder to interpret. 	We can control complexity using pruning, max depth, min samples per leaf, etc.	**Example** Shallow Tree (Easy to interpret)  Deep Tree (Harder to interpret)  Shallower trees are easier to understand and explain.

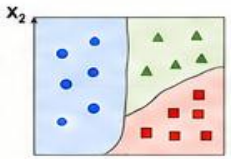
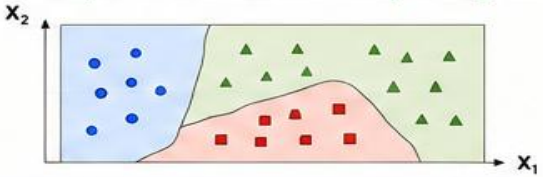
Key Takeaway

Decision Trees are highly interpretable because they use simple if-then rules, capture non-linear patterns and interactions, and provide clear reasoning for each prediction.

Transparent · Rule-based · Interpretable · Reliable

Explainable Properties of Rule Induction Models

Rule Induction models learn IF–THEN rules from data, making predictions easy to understand and explain.

Property	Values (Rule Induction)	Explanation	Example / Visual
Local or global Ruleset is global, and each rule explains local predictions.	**Global + Local** The entire set of rules is global; each rule (and its conditions) provides a local explanation for predictions. 	We can understand the overall decision logic (global) and also explain a single prediction using the rule that fired (local).	**Example: Credit Risk Rules** R1: IF Income > 50K AND Credit Score > 700 THEN Approve R2: IF Income <= 50K AND Debt <= 20K THEN Approve R3: IF Credit Score <= 600 THEN Reject R4: IF Debt > 40K THEN Reject ELSE Review The ruleset is global. For a new applicant, the matched rule explains the decision (local explanation).
Linear or non-linear Can capture both linear and non-linear decision boundaries.	**Non-linear** Rules partition the feature space into arbitrary shapes. 	By combining multiple rules, we can model complex and non-linear relationships.	**Example: Non-linear boundaries captured by rules**  Decision regions can be any shape based on rules.
Monotonicity Monotonic relationships are not guaranteed but can be enforced when needed.	**Not necessarily monotonic** Predictions may increase, decrease or stay the same as a feature increases. 	A small change in a feature may not lead to a consistent increase/decrease in prediction.	**Example** Income (K): 20, 30, 40, 50, 60, 70, 80 Prediction: 0, 0, 0, 1, 1, 0, 1 Prediction is not monotonic as Income increases.
Feature interactions captured Interactions can be captured explicitly in rule conditions.	**Yes (explicitly)** Rules can include combinations of features (AND/OR) to represent interactions. 	A rule like "Income > 50K AND Credit Score > 700" captures the interaction between the two features.	**Example: Interaction** IF Income > 50K AND Credit Score > 700 THEN Approve The rule depends on both Income and Credit Score.
Model complexity Generally low to medium; depends on number and length of rules.	**Low / Medium** A small set of short rules is simple and easy to interpret; many or long rules increase complexity. 	Complexity increases with the number of rules and the number of conditions per rule.	**Example** Simple Ruleset (High Interpretability): R1 ...., R2 ...., R3 .... Complex Ruleset (Lower Interpretability): R1 ...., R2 ...., Rn .... Smaller ruleset is easier to understand.

Comparing Traditional Interpretable Algorithms

Algorithm	Best For	Explanation Form
Linear Regression	Continuous targets	Coefficients
Logistic Regression	Binary classification	Odds ratios
GLM	Non-normal targets	Coefficients + link fn.
GAM	Non-linear effects	Per-feature smooth plots
Naive Bayes	Text/categorical data	Per-feature log-prob.
Bayesian Networks	Causal/probabilistic	Graph + inference trail
Decision Trees	Mixed data, rules	Root-to-leaf path
Rule Induction	Compact rule sets	IF-THEN rules

Putting It All Together

- Model visualization techniques provide diagnostic interpretability – understanding model quality and behavior at a system level.
- Traditional interpretable algorithms provide intrinsic interpretability – transparency built into the model itself.

Common Pitfalls in Model Visualization

- Relying on a single aggregate metric (e.g., accuracy) without visual diagnostics can hide critical failure modes.
- Misreading ROC curves under heavy class imbalance – prefer PRC in such cases.
- Ignoring influential points (Cook’s distance) that may make a regression model appear better or worse than it truly is.
- Choosing cluster count purely by elbow method without validating with silhouette analysis.

Tools Ecosystem Summary

- Yellowbrick: unified visual diagnostics for classification, regression, and clustering.
- scikit-learn: core algorithms, metrics, and validation utilities.
- statsmodels: detailed statistical summaries for linear/GLM models (p-values, confidence intervals).
- pyGAM: fitting and visualizing generalized additive models.
- pgmpy: construction and inference for Bayesian networks.